



WYDZIAŁ ELEKTRONIKI,  
TELEKOMUNIKACJI  
I INFORMATYKI

Imię i nazwisko studenta: Wiktor Czetyrbok  
Nr albumu: 188695  
Poziom kształcenia: studia drugiego stopnia  
Forma studiów: stacjonarne  
Kierunek studiów: Informatyka  
Specjalność: Inżynieria systemów informacyjnych

## **PRACA DYPLOMOWA MAGISTERSKA**

Tytuł pracy w języku polskim: Analiza porównawcza wydajności, skalowalności i kosztów utrzymania architektury monolitycznej z architekturą opartą na mikroserwisach w środowiskach chmurowych

Tytuł pracy w języku angielskim: A comparative analysis of the performance, scalability, and maintenance costs of monolithic and microservice architectures in cloud environments

Opiekun pracy: dr Adam Przybyłek

## **Streszczenie**

Celem pracy była analiza porównawcza architektury monolitycznej oraz architektury mikroserwisowej pod kątem ich wydajności, skalowalności oraz kosztów utrzymania w środowiskach chmurowych. W dobie rosnącej złożoności systemów informatycznych oraz dynamicznego przyrostu liczby użytkowników coraz większe znaczenie zyskują modele architektoniczne umożliwiające efektywne zarządzanie zasobami, elastyczność oraz niezawodność. W pracy zbadano, w jaki sposób wybór architektury wpływa na realne parametry działania aplikacji, a także jakie konsekwencje niesie dla organizacji pod względem operacyjnym i finansowym.

Przeprowadzony przegląd literatury pozwolił zidentyfikować aktualne podejścia do pomiaru wydajności i skalowalności systemów rozproszonych oraz istniejące narzędzia benchmarkingowe. Jednak większość dostępnych badań analizuje architektury w oderwaniu od praktycznego kontekstu wdrożenia w chmurze, co wskazało na istnienie luki badawczej — brak kompleksowego porównania monolitu i mikroserwisów uruchomionych w porównywalnym środowisku, z zastosowaniem jednolitych metod obciążenia i identycznych scenariuszy testowych.

W ramach pracy zaimplementowano aplikację benchmarkingową w dwóch wariantach architektonicznych — monolitycznym oraz mikroserwisowym — oraz wdrożono oba modele na platformie chmurowej. Zastosowano testy obciążeniowe generowane za pomocą dedykowanego narzędzia oraz przeprowadzono analizę skalowania pionowego i poziomego. Dodatkowo zbadano wpływ konfiguracji infrastruktury, mechanizmów komunikacji, wielowątkowości oraz sposobu zarządzania zasobami na ogólną efektywność aplikacji.

Wyniki badań wskazują, że architektura monolityczna charakteryzuje się wyższą wydajnością przy niewielkim obciążeniu oraz niższymi kosztami utrzymania w prostych instalacjach, natomiast architektura mikroserwisowa oferuje znacząco lepszą skalowalność poziomą, niezależność komponentów oraz większą elastyczność w zakresie optymalizacji wybranych obszarów systemu. Jednocześnie mikroserwisy wprowadzają koszty związane z komunikacją między usługami, złożonością infrastruktury oraz większym nakładem na utrzymanie.

Praca kończy się omówieniem ograniczeń badania, wkładu pracy w obszar inżynierii oprogramowania oraz propozycją kierunków dalszych badań, obejmujących m.in. analizę architektur hybrydowych, automatyczne skalowanie zasobów oraz wpływ technologii serverless na koszty operacyjne aplikacji.

**Słowa kluczowe:** Informatyka, Mikroserwisy, Monolit, Analiza kosztów, GCP, gRPC, Aplikacja webowa

**Dziedzina nauki i techniki, zgodnie z wymogami OECD:** Nauki o komputerach i informatyka

## **Abstract**

The aim of this thesis was to conduct a comparative analysis of monolithic and microservice architectures in terms of performance, scalability, and maintenance costs in cloud environments. As software systems become increasingly complex and the number of users continues to grow, architectural models that provide effective resource management, flexibility, and reliability gain significant importance. This work investigates how the choice of software architecture influences real application performance parameters and what operational and financial consequences it entails for an organization.

A systematic literature review was carried out to identify current approaches to measuring the performance and scalability of distributed systems, as well as existing benchmarking tools. However, most available studies examine these architectures without a practical cloud deployment context, exposing a research gap — the lack of a comprehensive comparison of monolithic and microservice applications deployed in equivalent environments, using unified load-testing methods and identical test scenarios.

As part of the thesis, a benchmarking application was implemented in two architectural variants — monolithic and microservice — and both versions were deployed on a cloud platform. Load tests were conducted using a dedicated tool, and vertical and horizontal scaling were analyzed. Additionally, the impact of infrastructure configuration, communication mechanisms, multithreading, and resource management strategies on overall application efficiency was examined.

The study results indicate that monolithic architecture provides higher performance under low load and lower maintenance costs in simpler deployments. In contrast, microservice architecture offers significantly better horizontal scalability, component independence, and greater flexibility in optimizing specific parts of the system. At the same time, microservices introduce additional costs related to inter-service communication, infrastructure complexity, and operational overhead.

The thesis concludes with a discussion of study limitations, the scientific contribution of the work, and directions for further research, including hybrid architectural models, automatic resource scaling, and the impact of serverless technologies on the operational costs of cloud applications.

**Keywords:** Computer Science, Microservices, Monolith, Cost Analysis, GCP, gRPC, Web Application

**OECD Field of Science and Technology Classification:** Computer and Information Sciences

## TABLE OF CONTENTS

## **LIST OF ABBREVIATIONS**

- app - application
- keto - Ketogenic diet
- carbs - Carbohydrates
- RE - Requirements Engineering
- IT - Information Technology
- DB - database
- GCP - Google Cloud Platform
- API - Application Programming Interface
- ID - Identification
- UI - User Interface

# 1. INTRODUCTION

## 1.1. *Motivation, goal of the project, research question*

AGNIESZKA KULETA, KAMIL DĘDZA

Food is an intrinsic part of our lives, without it, we simply cannot survive. We eat not only to provide our bodies with the essential nutrients they need, but also because food brings us some kind of pleasure. But what if your body reacts badly to certain substances? The answer is straightforward: we can stop consuming them. This approach works well if you need to avoid just one or two ingredients, we just eliminate certain meals from our diet. But what happens if you have to give up several of your favorite ingredients and you do not know how to substitute them so the taste of your favorite meals remains unchanged? Would you completely change your diets without any substitutes of your favorite dishes? Again everyone will answer that they would start to look for some substitutes or completely new recipes. However, finding them is not that easy.

Nowadays, the number of recipes available for people with food restrictions is quite limited and the number of people with many specific food allergies [1] and specific diets is still growing as shown on the Figure ??, [2].

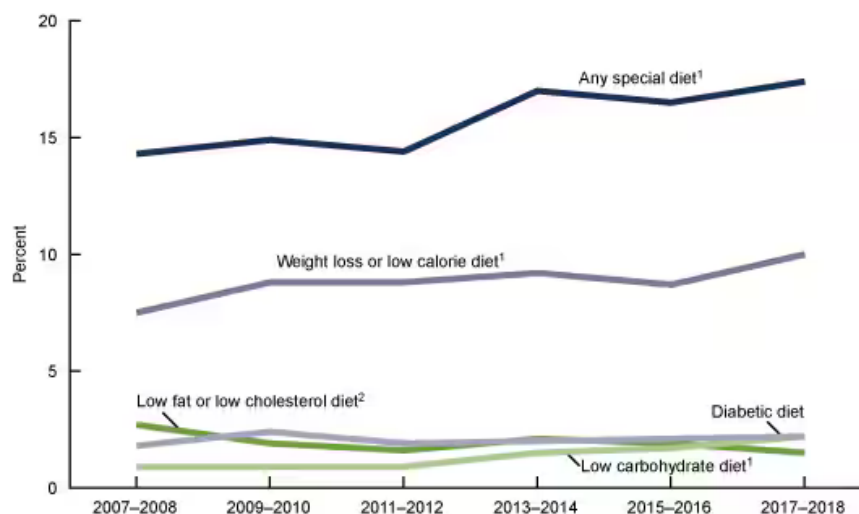


Fig. 1.1. Trends in the use of special diets among U.S. adults, 2007–2018 [2].

People, who are forced to follow the special diet, often eat only what they can, not what they want, because finding suitable substitutes is challenging, [3] without going through thousands of books or blogs. These people are just tired of trying new things. This lack of accessible and varied recipes limits their dietary options, making it difficult to enjoy eating and could cause serious health issues [3]. Here raises the question: How can we add variety to our diet and eat like everyone else without showing any signs of helplessness, dissatisfaction or serious health problems?

As you can clearly see that there is a growing need for more resources and creative solutions to the culinary world. Our culinary portal “Foodies” is created to fill this gap. Our aim was to create a web application, which is not only a place for suggestions of the next breakfast or dinner, but also a help in managing your specific diets and everyone’s needs by eliminating all unwanted ingredients and substituting them with one which you can and like to eat.

Existing solutions available on the market are mainly concentrated on the recipes itself, with little to no help for people with more specific needs. As you can see in further research ??, none of these solutions provides directly different substitutions of ingredients. Some portals write the substitute for one ingredient, mostly to substitute meat’s product, but the substitute is not included in the number of calories or other nutrition. Furthermore, changing wheat or dairy products was not the option. To find some substitutes, they need to search many different sources, which is time consuming and inconvenient. Moreover, not even all portals provided the numbers of calories or nutrition in the recipe ?? .People want to eat what they can and like and want to know how much they eat.[4] So, the goal of our project is to enable them not to give up what they want to eat, only because some of the ingredients do not match theirs’ diet.

For many years, our whole team has been interested in nutrition, each of us with a different perspective. Cooking, calories intake, meal preparation to name a few, are topics that our group is familiar with. Combined, we form a solid team to put our expertise for a good use. The project will focus on the users with specific diets, which are becoming more and more popular nowadays. The app will let them personalize their searches with the type of the diet they are currently following, rate and comment these recipes and personalize their profiles.

## **1.2. Project organization**

AGNIESZKA KULETA

Having said about our motivation to do this project, let’s talk about implementation. The implementation of the project is divided in three key stages: business analysis, implementation, test and verification.

The business analysis stage started from setting our goal and our motivation of doing this project, as was outlined in this chapter. We gathered all requirements, which will be detailed in chapter ??. Additionally, we chose a project development methodology, unanimously deciding on **Scrum**, due to its iterative and collaborative framework. After establishing tools, team responsibilities and initiating our Backlog, which we will describe more comprehensively in the chapter ??.

Completing business stage brought us to implementation stage, which began with data collection and database design, described in detail in chapter ??. After dealing with data, we moved on prototyping our app to verify established requirements and gain insights how we want our app to look like. Further details about whole architecture and its implementation might be found in chapter ??.

The final stage testing is documented in chapter ??. This crucial phase allowed us to draw the final conclusions.

## 2. RESEARCH ANALYSIS

### 2.1. Literature research

AGNIESZKA KULETA

#### 2.1.1. Needs for specific diet growing

In recent years, there has been a notable increase in people following specific diets[2]. For some, dietary restrictions are necessary to manage health issues or food sensitivities, while others adopt these diets to lose weight, build muscle, or reduce their environmental impact.

There are various reasons why we as a population decide to take the effort of being on diet. According to **US National Center for Health Statistics**[2] this need increased with increasing each person obesity index and increasing educational level. What's why Americans participating in study usually have chosen diets connected to weight loss. They were choosing mostly low carbohydrate or low calories diets. Their aim was really simple to lose weight. However, for younger generations this is not enough, now GEN Z is taking step forward [5]. They are introducing the training and healthy eating as a lifestyle. They love to lift weights and eat proteins [6]. For them our portal would be also very helpful as we marked it in the Persona model which will be fully presented in "*Requirements*" chapter. Let's say that it will be our first cause that provides the need of diets.

The second, and for us the most important, reason for creating this app is to support people who must follow specific diets due to food allergies. According to **World Health Organization** [7], globally more than 220 million people are forced to avoid certain foods because of allergies. There are really limited protection of individuals with food allergies because the only solution is to strictly adhere to restrictions. The serious health risks done by allergens and the limited treatment options available highlight the importance of knowing which foods are most commonly associated with severe allergic reactions. Moreover, according to online side **Food Allergy Research and Education**[8] having a food allergy heavily impacts the quality of life, not only the person who has it, but also their relatives. The disease results in exclusion of children from school canteens and prevents their full participation in school life and society. It also is written that, mothers of food-allergic children under age five have significantly higher blood-pressure measurements than mothers whose children do not have food allergies. That is also why we took one mum as a Persona model which will be fully presented in "*Requirements*" chapter. That mum does not know how to deal with food allergy of her child and needs application to gain the ideas of new recipes. Mum also tries to cook for a whole family as close to "normally" as possible, without the constant worry of accidental exposure to allergens. That's why in our application we will take into account gluten-free and no-dairy diets, because they are two most common and the hardest to avoid allergens on the daily basis.

Finally, the last but not least reason why people want to change the way of eating are health, environment and morality [9]. After a quick glance at the websites [9] of vegan and



vegetarian organizations gives three mentioned reasons are the answer to the question why people do adopt a plant-based lifestyle. This shift is more than a dietary change; it represents a comprehensive transformation in mindset. The approach to meat consumption among vegans and vegetarians shows minimal difference during the initial stages, as illustrated on Figure??.



Fig. 2.1. Motives not to eat meat vegans vs vegetarians [10].

From the chart we might see that people who are vegan and vegetarians similarly value Health and Environment factors. However, animal rights show a difference. They are more important to vegans than to vegetarians. Such a distinction suggests that while both diets are plant-based, the motivation behind a vegan lifestyle may lead to stricter dietary choices focused on minimizing harm to animals.

For each diet we would like to give you quick overview by highlighting what should be eat and what should be avoided. Of course, we are fully aware that we will not describe and mark every existing diet, we just want to give you the main idea of each diet included in our application. To make it far easier and faster to get to know strict rules and restriction for each diet without any details and insights, we have prepared Table ??.

|                | VEGAN | VEGETARIAN | OMNIVORE | LACTOSE -ALLERGIC | GLUTEN-ALLERGIC | KETO |
|----------------|-------|------------|----------|-------------------|-----------------|------|
| MEAT           |       |            | X        | X                 | X               | X    |
| EGGS           |       | X          | X        | X                 | X               | X    |
| WHEAT AND OATS | X     | X          | X        | X                 |                 | X    |
| DAIRY          |       | X          | X        |                   | X               | X    |
| VEGETABLES     | X     | X          | X        | X                 | X               |      |
| FRUITS         | X     | X          | X        | X                 | X               |      |
| SOY            | X     | X          | X        | X                 | X               | X    |
| PEANUTS        | X     | X          | X        | X                 | X               | X    |

Table 2.1. Products categories included in each diet

#### 2.1.1.1. Obesity and fit trend reason

While contemporary fitness and weight management trends often prioritize calorie counting and macro-elements tracking, there is less focus on the diversity of food. To follow these type of diets the key is calories counting and nutrition elements counting.

- **Low calories**

The main idea of this diet is too feel full on fewer calories. Achieving this goal often requires meticulous calorie counting for each meal. Tracking calorie intake helps ensure you stay within a set daily limit and allows for better planning of subsequent meals to meet those goals. According to the Mayo Clinic [11], choosing the low-calories diet for most adults typically reduces daily intake to between 1,200 and 1,800 calories, though this range may be too restrictive for some, depending on their health status and history. Rather than focusing on specific foods to eat or avoid, the key is consistently sticking to your daily calorie limit.

- **Ketogenic diet**

A ketogenic diet is a low-carbohydrate, high-fat diet, focuses on drastically reducing carbohydrate intake while increasing fats to put the body into a state called ketosis. In ketosis, the body burns fat for energy instead of carbs. In order to trigger ketosis, the carbs you eat need to be heavily restricted – down to no more than 20-50 grams per day. According to **Health direct in Australia** [12], it is proved that it works for long term weight lost. However not everyone can use it for example it is really bad for children. The main goal of this diet is to reduce carbs. So the main foods which is included a ketogenic diet are high quality fats like:

- avocado
- olive oil
- coconut oil
- nuts and seeds

Also, every kind of meat or fish source and eggs, which is not only full of fats but also proteins. You may have some low carbohydrate vegetables, such as lettuce and cucumbers. On a typical ketogenic diet, you will have only small amounts of rice, pasta, fruit, grains, bread, beans and starchy vegetables such as peas and potatoes, because all of these ingredients are full of carbs. And of course, you should not eat: fruits, preprocessed sweets and snacks like chips, drinking alcohol, sugar. In the Table ??, it is establish which products are worth eliminating and what can be eaten in exchange [13] [14].

| To Eliminate | Substitute                               |
|--------------|------------------------------------------|
| Wheat        | Almond flour, Coconut flour              |
| Rice         | Cauliflower rice, Shirataki rice         |
| Sugar        | Erythritol, Stevia, Monk fruit, Allulose |
| Bananas      | Strawberries                             |
| Apples       | Raspberries                              |
| Oranges      | Blackberries                             |
| Butter       | Avocado                                  |
| Pasta        | Zucchini noodles                         |

Table 2.2. Keto diet ingredients to eliminate and their substitutes

#### 2.1.1.2. Allergenic reasons

Firstly, we might wonder why allergies are so frustrating - why they are reasons to limit ourselves? An allergy is a health response triggered by a specific immune reaction to certain foods. These dietary restrictions are increasingly common, driven by the rising number of food allergies. But what foods commonly cause these reactions? Would not it be simpler if only a single ingredient were to blame?

According to U.S. Department of Agriculture [15], allergic reactions might be caused by more than 160 foods, however there are 9 most common items established by them, which account for 90 percent of food allergic reactions. There are called BIG9:

- milk
- eggs
- fish
- shellfish
- tree nuts
- peanuts
- wheat
- soybeans
- sesame

Each restriction differ and it is not easy to deal with any of it. However, taking for an example wheat and peanuts what would be easier for you to eliminate? Or wheat and eggs? What do you choose? Now, imagine being unable to consume either lactose or gluten. What could you eat then? Would you have to rely solely on a vegan, gluten-free diet? In this chapter, we will focus on gluten-free and dairy-free diets, as they pose some of the greatest challenges due to the widespread presence of these ingredients in typical European meals.

#### 2.1.1.3. Gluten-free diet

First question crossing our mind is what exactly gluten is? According to **Celiac Disease Foundation**[16] is a type of protein found in grains such as wheat, barley, rye and oats. Sensitivity to gluten is a symptom of three main disorders: celiac disease, wheat allergy, and non-celiac gluten sensitivity. All of these illnesses have common symptoms like diarrhea, stomach pain, and vomiting. Unfortunately, the only way to improve health and avoid all the symptoms is making careful dietary choices and avoid foods made from gluten-containing grains. But what type of food contains it? Unfortunately we have quite big amount of food to avoid to: breads, pastas/noodles, cereals, dumplings, soy sauce, fried/breaded foods, pastries, cake, crusts, cookies, crackers, tortillas, gravy/thick sauces, beer. And now let's see how do we can substitute it? The answer to this question can be found in the Table ???. This table was prepared based on blogs [17][18][19] and own experiences.

| TO ELIMINATE                 | SUBSTITUTE                                                 |
|------------------------------|------------------------------------------------------------|
| COUSCOUS                     | CAULIFLOWER                                                |
| CORNSTARCH                   | ARROWROOT                                                  |
| OATS                         | GLUTEN FREE OATS                                           |
| BREAD CRUMBS                 | ALMOND MEAL                                                |
| PASTA                        | ZUCCHINI, CHICKPEA PASTA, QUINOA PASTA, CORN PASTA         |
| LOLLIES                      | NUTS                                                       |
| SOY SAUCE                    | TAMARI SAUCE                                               |
| MALT VINEGAR                 | BALSAMIC VINEGAR                                           |
| NOODLES                      | SHIRATAKI NOODLES, RICE NOODLES                            |
| CEREALS                      | GLUTEN-FREE CEREALS                                        |
| SOY SAUCE                    | COCONUT AMINOS, TAMARI SAUCE                               |
| BREADCRUMBS                  | CRUSHED CORNFLAKES, RICE CRACKERS, GLUTEN-FREE BREADCRUMBS |
| TORTILLAS                    | CORN TORTILLAS                                             |
| BEER                         | HARD CIDER                                                 |
| FLOUR                        | COCONUT FLOUR, RICE FLOUR, ALMOND FLOUR                    |
| BULGUR                       | RICE, BUCKWHEAT, QUINOA                                    |
| COUSCOUS                     | QUINOA                                                     |
| FARINA                       | CREAM OF RICE                                              |
| GRAHAM FLOUR                 | ALMOND FLOUR                                               |
| HYDROLYZED VEGETABLE PROTEIN | SOY PROTEIN ISOLATE                                        |
| SEITAN                       | TEMPEH, JACKFRUIT, TOFU                                    |
| WHEAT BERRIES                | QUINOA, BROWN RICE                                         |
| WHEAT GERM                   | CHIA SEEDS                                                 |
| MALT                         | BROWN RICE SYRUP                                           |
| BROWN RICE SYRUP             | MAPLE SYRUP, HONEY                                         |

Table 2.3. Gluten-free diet forbidden products and their substitutes

#### 2.1.1.4. Dairy-free diet

Typical European diet contains yogurt, butter or cheese made from the milk of animals including that from cows, goat or sheep. However, many people are unable digest the lactose in milk. According to **Arizona Digestive Health** [20], it is estimated that approximately 65 percent of the world's population is lactose intolerant. However, it is only one from many reasons, people also can have milk-allergy. For these people eating products which contain lactose cause the allergic reaction by causing symptoms like discomfort, stomach pain and diarrhea. There are no other cure than diet. So these people have to follow restrictions of the diet, which is avoiding lactose—a type of simple sugar found naturally in milk. This type of diet avoids all milk-derived ingredients, including obvious ones like cheese, butter, cream, and yogurt, as well as less obvious sources such as whey, casein, and certain baked goods. To successfully maintain a dairy-free diet, let's have a look at the Table ???. This table was prepared based on blogs [21][22] and own experiences.

| TO ELIMINATE      | SUBSTITUTE                                                  |
|-------------------|-------------------------------------------------------------|
| Acid Casein       | Soy protein isolate, pea protein isolate                    |
| Cheese            | Dairy-free cheese slices (e.g., made from coconut oil, soy) |
| Asiago            | Cashew cheese                                               |
| Brie              | Cashew                                                      |
| Butter            | coconut                                                     |
| Butter Oil        | Coconut oil                                                 |
| Buttermilk        | apple cider vinegar                                         |
| Camembert         | Cashew                                                      |
| Caramel           | Coconut milk caramel                                        |
| Condensed Milk    | soy milk                                                    |
| Cream             | Coconut cream, oat cream, or cashew cream                   |
| Feta              | Tofu feta, almond-based feta                                |
| Fontina Cheese    | Plant-based cheese with a mild flavor (e.g., cashew cheese) |
| Gorgonzola        | Marinated tofu                                              |
| Kefir             | Coconut kefir                                               |
| Low Fat Milk      | Almond milk, oat milk, or soy milk                          |
| Ricotta           | Almond ricotta, tofu ricotta, or cashew ricotta             |
| Yogurt            | Coconut yogurt, almond yogurt, soy yogurt                   |
| Whole Milk Solids | Coconut milk powder, soy milk powder                        |
| Whey Peptides     | Pea peptides, rice protein peptides                         |

Table 2.4. Dairy-free diet forbidden products and their substitutes

#### 2.1.1.5. Environment and morality

Having said, why people usually switch to plant-base diet, let's have a look the differences between vegans and vegetarians. According to study made by Kent University [10] more than 80 percent of studied population switched to vegan diet, because of Animal ethics and comparing to vegetarians this reason is a little bit less popular. Between this two groups there is some disagreement as we can see on the Figure??.

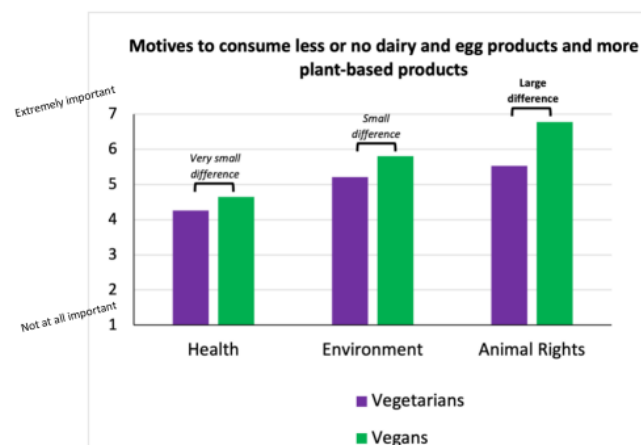


Fig. 2.2. Differences between vegans and vegetarians approach[10].

This chart reflects not only psychological differences between this two groups but also explains the differences in diets. Explains why vegetarians only do not eat animals, but they eat the products which come from an animal, and vegans do not. The products, which are avoided by vegans and their substitutes, are in the Table ???. There are also marked in pink, products which are avoided by vegetarians too.

| TO ELIMINATE           | SUBSTITUTE                                               |
|------------------------|----------------------------------------------------------|
| Pig                    | Tofu, Seitan                                             |
| Beef                   | Tempeh, Jackfruit                                        |
| Chicken                | Oyster mushrooms, Tofu                                   |
| Veal                   | Mushrooms                                                |
| Fish sticks            | Vegan fish sticks                                        |
| Cod                    | Banana blossom                                           |
| Shrimps                | Tofu                                                     |
| Crabs                  | Hearts of palm                                           |
| Salmon                 | Artichoke, Seaweed                                       |
| Tuna                   | Soy watermelon                                           |
| Milk                   | Rice milk, Coconut milk, Oat milk, Soy milk, Almond milk |
| Cheese                 | Cashew cheese, Almond-based cheese, Soy cheese, Violife  |
| Butter                 | Avocado oil, Coconut oil                                 |
| Yogurt                 | Oat-based/ Coconut-based /Soy-based /Almond-based yogurt |
| Cream                  | Soy-based cream,Cashew cream                             |
| Eggs (scrambled)       | Chickpea flour scramble                                  |
| Eggs (baking)          | Chia seed, Applesauce                                    |
| Honey                  | Agave nectar, Date syrup, Brown rice syrup, Maple syrup  |
| Gelatin                | Carrageenan, Pectin, Agar-agar                           |
| Lard                   | Vegan margarine, Coconut oil, Vegetable shortening       |
| Carmine (red coloring) | Beet juice                                               |
| Casein and Lactose     | Soy protein                                              |
| Shellac                | Vegan-friendly coatings                                  |

Table 2.5. Plant-based Alternatives to Common Animal Products

Having said about all diets, which were detailed analyzed and explained, it is time to focus on our main goal of the app. Our main requirement is to substitutes disliked products or allergens. So we have also done the research how we can substitute every ingredient in our app for different products. When we were choosing substitutes we were looking for thousands of blogs and we were testing some recipes at home to find the right substitutes, which were similar in texture and flavor to original products.

## 2.2. Existing systems

AGNIESZKA KULETA

After discussing various diets and the motivations behind dietary restrictions, we did necessary overview to determine whether any existing applications align with our core ideas and concepts. Our main ideas and needs were outlined in the "Motivation" section, the complete system functionalities will be detailed in the "Requirements" chapter.

Table ?? highlights the functionalities of popular culinary platforms, especially familiar to Polish students. Our goal was to identify which platform, if any, could meet the requirements set by our stakeholders. Unfortunately, none of the reviewed platforms did so. It might be easily concluded by making comparative analysis presented in the table. This analysis highlights the uniqueness of our functionalities and the need to implement our solution.

Further details on the specific requirements and functionalities, design and implementation will be provided in the following chapters.

|                                                                           | aniagotuje.pl                             | https://www.kwestiasmaku.com/           | https://www.jamieoliver.com/                        | Foodies                                                   |
|---------------------------------------------------------------------------|-------------------------------------------|-----------------------------------------|-----------------------------------------------------|-----------------------------------------------------------|
| LOG IN                                                                    | y                                         | y                                       | y                                                   | y                                                         |
| SAVING RECEPIES                                                           | y                                         | y                                       | y                                                   | y                                                         |
| ADDING NOTES                                                              | y                                         | y                                       |                                                     |                                                           |
| SEARCHING FOR RECIPIE                                                     | y                                         | y                                       | y                                                   | y                                                         |
| ADDING COMMENTS                                                           | y                                         | y                                       |                                                     | y                                                         |
| GIVING RATING                                                             | y                                         | y                                       |                                                     | y                                                         |
| CATEGORIES (LIKE DINNER, BREAKFAST)                                       | y                                         | y                                       | y                                                   | y                                                         |
| PRINTING OF RECEPIES                                                      | y                                         | y                                       |                                                     |                                                           |
| SELF CALCULATING MACROELEMENTS                                            |                                           |                                         |                                                     | y                                                         |
| GIVEN MACROELEMENTS                                                       | y                                         | y                                       | y                                                   | y                                                         |
| ADDING FOTO IN COMMENTS                                                   |                                           | y                                       |                                                     | y                                                         |
| NEWSLETTER SUBSCRIPTION                                                   | y                                         |                                         | y                                                   |                                                           |
| COOK MODE (KEEP SCREEN AWAKE)                                             | y                                         |                                         |                                                     | y                                                         |
| COPY ALL INGREDIENTS BY ONE CLICK                                         | y                                         |                                         |                                                     |                                                           |
| SORTING                                                                   | publication date and popularity           | publication date and popularity         |                                                     | by number of calories, by rating                          |
| TAGS                                                                      | y                                         |                                         | y                                                   |                                                           |
| FILTERS                                                                   | by diets : gluten free, vegan, vegetarian |                                         |                                                     | by number of calories, by diets                           |
| DIETS                                                                     | omnivore, glutenfree, vegan, vegaterian   | omnivore, glutenfree, vegan, vegaterian | omnivore, glutenfree, vegan, vegaterian, diary free | omnivore, glutenfree, vegan, vegaterian, diary free, keto |
| MACROELEMNTS MORE DETAILED(NOT ONLY CALORIES)                             | y                                         |                                         | y                                                   | y                                                         |
| POSSIBILITY FOR EDITING USER'S DATA                                       |                                           | y                                       |                                                     | y                                                         |
| CULINARY CALCULATOR (IE. HOW MANY GRAMS IS A SPOON)                       |                                           | y                                       |                                                     |                                                           |
| BACKING CALCULATOR (IE. HOW MANY SMALL FRAMES TO USE INSTEAD OF BIG ONE)? |                                           | y                                       |                                                     |                                                           |
| DISLIKED INGREDIENTS LIST                                                 |                                           |                                         |                                                     | y                                                         |
| MODIFYING RECEPIES                                                        |                                           |                                         |                                                     | y                                                         |
| SAVING MODIFIED RECEPIES                                                  |                                           |                                         |                                                     | y                                                         |
| SUBSTITUTING INGREDIENTS IN RECIPIE                                       |                                           |                                         |                                                     | y                                                         |

Table 2.6. Comparison of existng solutions

### 3. REQUIREMENTS

#### 3.1. *Introduction*

WIKTOR CZETYRBOK, KAMIL DĘDZA

Creating new products always begins with a clear idea, but even more importantly, with well-defined requirements. These requirements specify what needs to be done, by whom, by when, and with which resources. In the real estate field for example, according to International Journal of Real Estate and Land Planning by Athanasios Lamprou and Dimitra Vagiona [23], the most important criteria for the project success is time and budget, but it is also worth to mention that high up in the hierarchy is also business and commercial performance as well as technical specifications and requirements. Both contribute to the importance of the requirements, and the same is within IT projects. In fact, over a half of the projects were challenged, meaning exceeded resources or lacking initially stated functionalities. Only 29% are fully successful, whereas 19% fail completely [24].

'Clarity is underappreciated as a requirements specification quality attribute. We studied the clarity of requirements forms, operationalized as ease of problem detection, least obstructive to understanding, and understandability by stakeholders' [25]. Not only the process of gathering the requirements is critical, but also how they are stated. The language is not always straightforward, and not always provides the same meaning for every word. The understandability of the particular requirement is also a big concern.

One can ask, how to make the proper requirements? The first hint is to involve the customers and users during the requirements engineering process. It means that it is important to collaborate with the stakeholders and leads to 'better understanding of real needs' [26]. The other best practice is to ensure prioritization of the particular requirements. One way to achieve this is creating a field, where there is possibility to keep the level of importance of the particular tasks. One should not forget about allocating about 20% of resources to the process of gathering the requirements [26]. This may feel like the resources are not efficiently allocated, but it will pay off in the future stages.

Having mentioned why requirements are not so straightforward for everyone, and what are the characteristics of accurate requirements, now the question arises what are the consequences of poorly gathered ones. Study from 2015 "Leading reasons for software project failure worldwide 2015" published by Statista Research Department [27] shows surprising results. The reason for almost half (48%) of the failures are the changing or poorly documented requirements. This statistic underscores how a lack of clarity or shifting expectations can disrupt the entire project lifecycle, leading to misunderstandings, misaligned objectives, and ultimately, an end product that fails to meet user needs. Accurate requirements gathering is not merely a preliminary task but a critical phase that can define the project's success or failure. It is essential to invest the necessary time and resources upfront to capture a comprehensive, well-documented set of requirements to ensure alignment, efficiency, and a final product that truly meets stakeholder expectations.



Requirements are the true, real foundation of the project. On the other side it is also worth mentioning that the second most popular reason is underfunding or under-resourcing and the third poor team or organizational management.

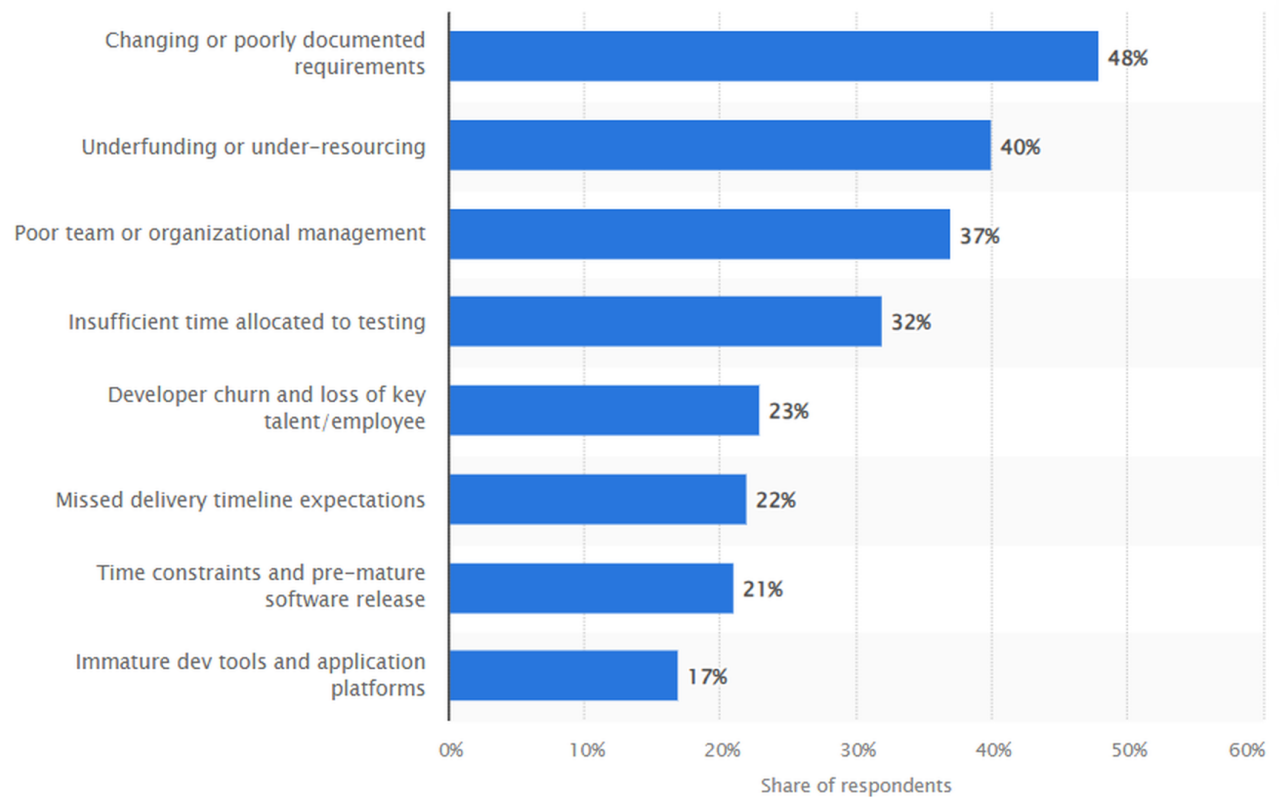


Fig. 3.1. Leading reasons for software project failure [27].

### 3.2. *How to gather requirements*

KAMIL DĘDZA

There are numerous techniques that are used during the process of gathering requirements. No single method suits all kinds of projects and is the golden rule that is used by every team. Each technique is tailored for uncovering specific information and suited for specific fields. For instance, a workshop, which is a meeting where many stakeholders, teams and experts can discuss and explore solutions, align on objectives and exchange ideas is facilitating the collaboration between all the parties. On the other hand there are interviews, where it is easy to dive deeper into the perspective of individuals and focus on their issue and their point of view. Observation is quite an interesting approach when it comes to gathering requirements. The process relies on looking at the users and making notes about their problems and interactions. This method suits best real-life usage and uncovering implicit requirements. Document analysis is also common, especially when there are a lot of reports and for instance the availability of the stakeholder is limited. There are many more techniques other than those mentioned above [28], but even after listing all of them there would still not be one that is much better than the other ones. 'A combination of RE tools/techniques are more effective'[29]. Obviously, the mix of different techniques, when those that are chosen piece together, increases the likelihood of

properly gathered requirements, hence also increases the likelihood of a solution that will be effective and suit the needs.

During the development of the application personas were created. It is a user-oriented approach for gathering requirements, which focuses on the needs and issues. By creating the representatives of groups of future users it is easy to capture their diverse point of view. Each persona is a fictional character that has its own name, background, profession, problems and specific needs. It helps the developers to understand their situation by providing a human approach to the requirements. During requirements gathering, teams leverage these personas to evaluate how different users might approach the system, what functionalities are critical to each, and what workflows need prioritization to best serve diverse user needs. Often it is the case that the people responsible for creating the solution are biased, and do not see or understand other groups of users. Focusing on personas ensures that the designed system will in fact resolve real user challenges, taking into account different groups with different requirements. The three personas that have been created are described by the short description of their background - who they are, benefits - what advantages they can take by using the system, goals and challenges - what they want to achieve and what is preventing them, personality - their character, information technology skills - how they are able to interact with a system and fears - worries with regards to current situation without the existing solution. Each of these characteristics makes it much easier and more understandable to create the requirements for the system that they will use in the future. The representatives differ from one other by far, yet still they are to be using the system. To overcome this challenge of not creating three different solutions, it is crucial to understand the needs of every group and try to incorporate them into one, final system.

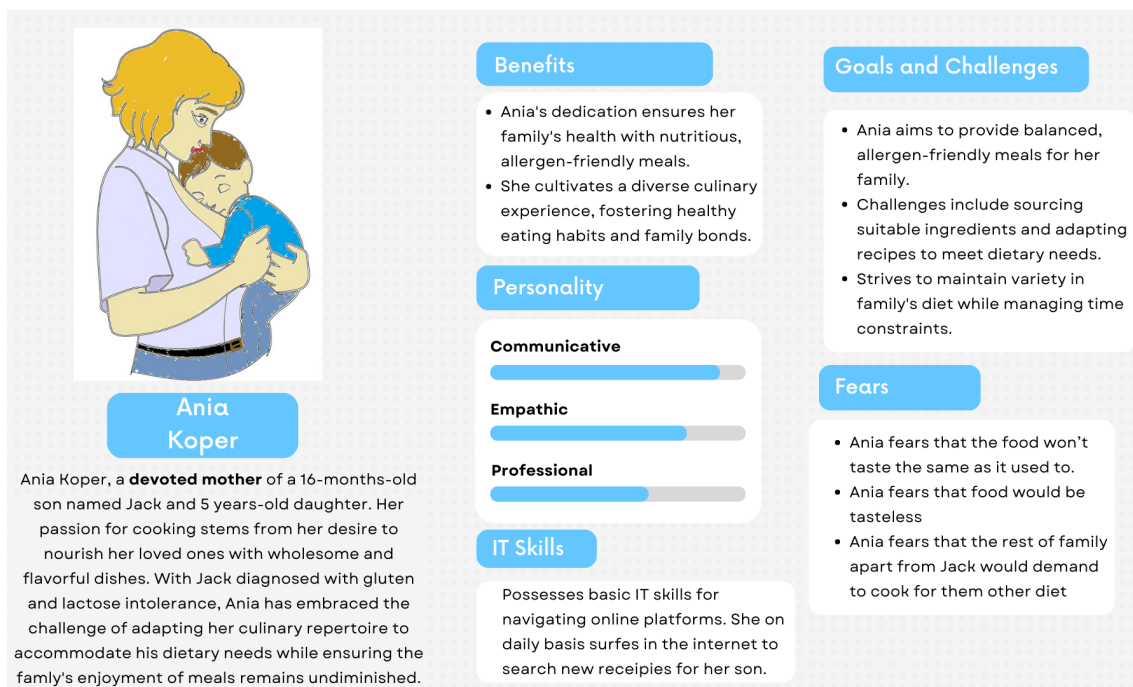


Fig. 3.2. Persona 1

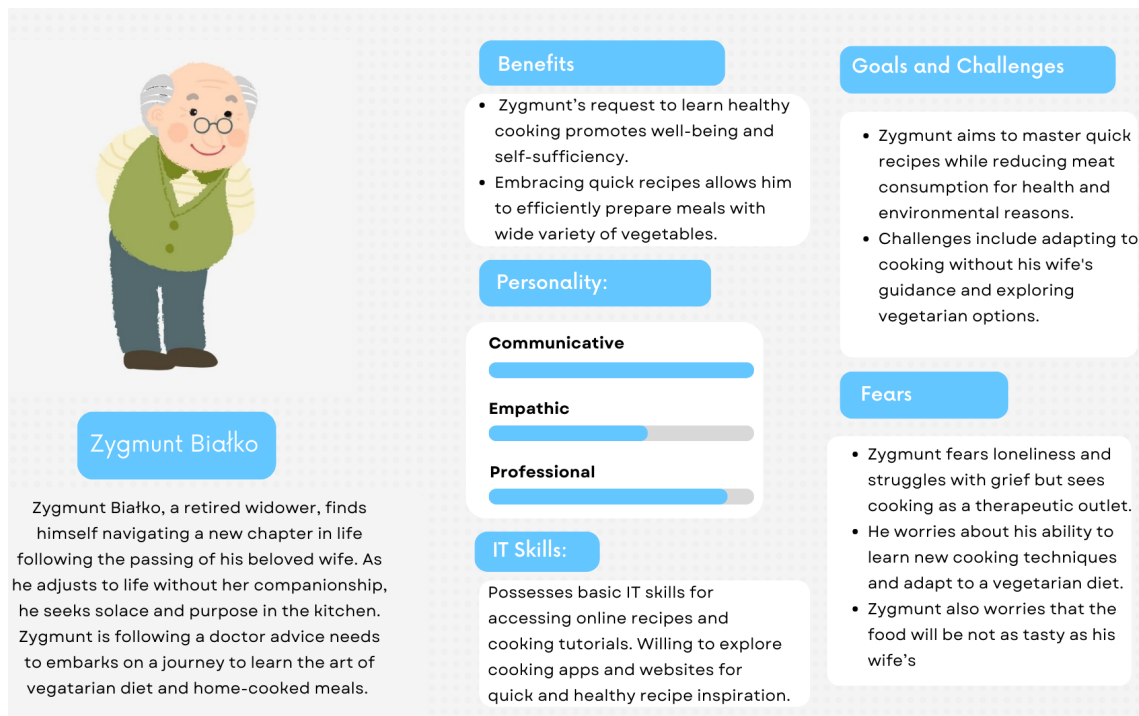


Fig. 3.3. Persona 2

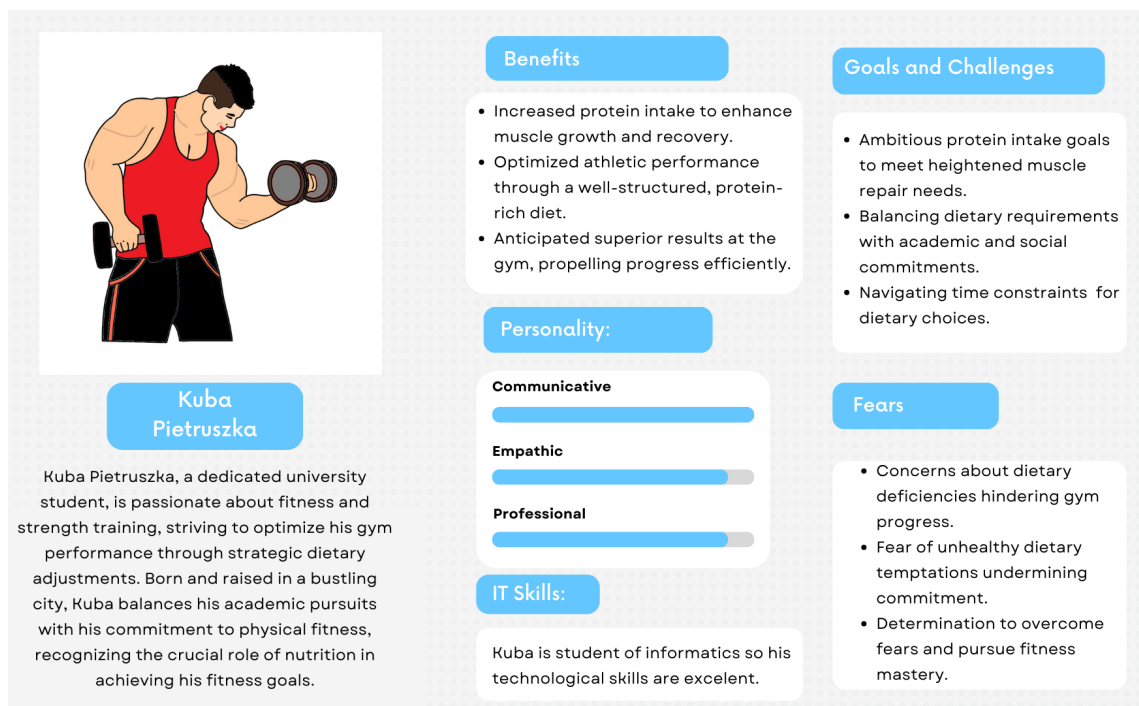


Fig. 3.4. Persona 3

'Most of the user story adopters (94 %) use them in combination with Scrum' [30], and is also a user-centered approach. The user stories are short, but the comprehensive descriptions can with ease capture the user needs in the form of a short story. The structure is simple, but

yet provides valuable insights into the requirements. They start with who is the speaker, which could be the previously created personas, what he or she wants to achieve, and what are the benefits [30]. Each user story identifies the specific, unique action, linking it with an outcome. One of the key takeaways from utilizing this technique is that it helps with breaking complex problems into many smaller chunks in the form of a short, typically one sentence long story. The second advantage is that this approach also presents the benefit of specific action, so that the developers can understand why particular needs are so important and what benefits can the users take from them. By creating the user stories as part of the requirements developers can maintain a connection to the users' needs. Last but not least, this method stimulates the iterative approach and prioritization of features that are actually strongly needed. There are many tools that support the creation of user stories, but Jira is the most popular and widely used. Everything is easily accessible by other team members, the stories can obviously have other fields apart from the description and name, but also the priority, assignee, estimate of work needed to accomplish, status, creation date just to name a few. These features keep the development organized, facilitate collaboration, and allow the team to adjust requirements flexibly throughout development, ensuring that the final product closely aligns with user expectations and agile principles.

Prototyping is an iterative and interactive technique used during the process of gathering the requirements. 'Prototyping is the art of creating and evaluating a simple version of a product whose purpose it is to bring clarity on a specific topic' [31]. It is a low fidelity, preliminary version of the designed system. Its goal is to present some basic features or workflows, and let user stakeholders interact with the intermediate solution. Then the prototype can evolve and become an even more realistic version of the final product. The tangible version makes it extremely easier to be grasped by others, because it solves the problem of ambiguity of for example text. It is not the easiest task to describe some features, but the visual representation is much more effortless. By interaction with the prototype, developers are able to detect the issues with the workflows. Quick feedback is what characterizes this approach. Low quality sketches or drawings presented to the stakeholders can immediately clarify the ideas and enhance interaction. This technique is great when it comes to solving the misunderstanding, because it provides a clear vision of what the goal is, and how the product is likely to look in the future stages. Market is full of tools ready to help with creating the prototypes. One of the most popular is Figma. It provides the features for clickable mockups and many more interactive elements, enhancing the quality of collaboration with the stakeholders.

## 4. METHODOLOGY

### 4.1. Team

**Project Tutor:** dr inż. Aleksandra Karpus

**Project Authors:**

- Wiktor Czetyrbok
- Kamil Dędzia
- Agnieszka Kuleta

#### 4.1.1. Organization of the Team

| Team Member      | Role               | Experience                        | Skills                             | Collaboration Type |
|------------------|--------------------|-----------------------------------|------------------------------------|--------------------|
| Kamil Dędzia     | Software Developer | Junior Data Engineer at DXC       | Python, PL/SQL, JAVA               | Remotely           |
| Wiktor Czetyrbok | Software Developer | Software Engineer at GridDynamics | JAVA, JavaScript, SQL, GCP, DevOps | Remotely           |
| Agnieszka Kuleta | Scrum Master       | Data Analyst at Nordea Bank       | MS SQL, Python, JAVA               | Remotely           |

Table 4.1. Team

#### 4.1.2. Project Responsibilities

- **Wiktor Czetyrbok:** Mainly responsible for application architecture and design.
- **Kamil Dędzia:** Mainly responsible for data management and integration.
- **Agnieszka Kuleta:** Mainly responsible for business requirements, communication with the product owner, and application design.

#### 4.1.3. Communication Within the Team

- **Communication Methods:** We communicate via email, chat, and video calls.
- **Communication Tools:** Messenger, Discord, Gdańsk University of Technology's Mail.
- **Meetings:** We meet at least twice a week, primarily using Discord.
- **Communication with Project Environment:** Every two weeks on Discord on Fridays at 2 PM (meetings with the product owner/tutor).

#### 4.1.4. *Sharing Code and Documents*

- **Code:** Available on GitHub (<https://github.com/culinary-portal>).
- **Documents:** Available on Google Drive.

## 4.2. *Scrum*

Scrum is an Agile methodology that through its iterative progress of short cycles **sprints** which last 2 weeks. It allows us to build new features, such as recipe database, user accounts, and interactive interfaces, while continually gathering feedback from stakeholders. It also enables us to respond to changing requirements, making sure that final product is closer to future users needs. This method also increases effectiveness of team communication and helps to prioritize tasks based on project goals.

### 4.2.1. *Backlog Documentation*

After creating a high level overview of all the requirements **??**, there is a necessity to introduce technical tasks. They are in fact a bridge between business requirements and detailed work of the developers. They answer the question how the team can achieve particular features, and how to implement them. Some stakeholders at first glance might not notice them, but they are crucial during the process of creating a solution. These types of product backlog items are also referred to as “technical debt” because, much like financial debt, they can cause problems if allowed to build up. It is wise to prioritize technical work alongside features and defects’ [32]. These tasks are usually written in clear language, what must be done under the hood, for example configuring the database or creating a communication between different services. They also help with complex problems, dividing them into smaller, separable chunks. The team of developers can in a clear way distribute the work between the members. Together, with previously mentioned business requirements they present the whole amount of work and resources needed to achieve the success and to create a solution. Planning can be also done only if the majority of requirements are gathered, and only then it is possible to estimate the time and work allocation. Additionally, this process offers individuals without extensive technical knowledge a clearer understanding of the complexity of certain features and the variety of tasks they involve, as outlined in at **Appendix A ??**.

The project backlog was created and managed using **Jira**, a software that enables the definition and prioritization of tasks. Each backlog item, such as epics, user stories, and tasks, was added to the system, providing easy access to detailed descriptions, statuses, and time estimates. In line with the scrum methodology, the organization of sprints played a crucial role in ensuring continuous progress, adaptability to changes, and frequent delivery of valuable increments. Every two weeks, our team defined goals and selected previously prioritized items based on the time and complexity of tasks, maintaining a structured and efficient workflow.

### 4.3. Tools

#### 4.3.1. Supporting tools

As supporting tools for areas such as backlog management, system development, and testing, we utilized a combination of platforms. Under the subsection of Work Organization, we incorporated tools such as **Jira** was used for tasks and backlog management, allowing the team to define, prioritize, and track tasks efficiently. For version control and collaborative development, we used **GitHub** and to host our services we have used **Google Cloud Platform**. **Google Drive** served as a centralized platform for storing and sharing project documentation, ensuring accessibility and organization of resources. Additionally, **Discord** was our primary communication tool for real-time discussions and coordination, complemented by **Email** via Outlook for formal communication. As a Programming tools we have used:

- Code Editors:
  - Visual Studio Code
  - IntelliJ IDEA
  - PyCharm
- Version Control git:
  - Git
  - Github
- Dependecy and Package Managment:
  - pip (Pip Installs Packages)
  - Gradle (Java)
  - npm (Node Package Manager)
- Database and management tools:
  - PostgreSQL
- Libraries and Frameworks:
  - Angular
  - Spring boot (Java)
- Programming Languages:
  - Python
  - Java
  - JavaScript
  - HTML, SCSS
- Web frameworks:
  - Spring (Java)
- Database:
  - PostgreSQL/Cloud SQL
  - Redis/Memorystore
- Frontend Technologies:
  - JavaScript
- Libraries and tools for building user interface:
  - Angular and Tailwind

## 5. DATA

### 5.1. *API/dataset used and relations between each of them*

KAMIL DĘDZA

Data is one of the most important parts of every IT solution. Our application is just the same. To start with we have found an open API (application programming interface), which serves our purposes. We needed the data about the different recipes, how they are prepared, and what ingredients are required. To start with, we found the themealdb [33]. It provides free of charge data and is openly accessible for use in personal, educational, and non-commercial projects. However, if the API is used as part of an application that is published commercially, it is expected to contact the authors [34]. The response provides many crucial fields for our application such as name of the recipe, ingredients or instruction on how to prepare the dish to name a few. Unfortunately, it does not come up with information about the nutritional values.

Hence, we also make use of nutritionix.com [35]. The Nutritionix API was and is being utilized in accordance with the proprietary rights and usage policies outlined by Nutritionix. As its creators stated, the API, along with the associated services, content, documentation, code, and data, remains the exclusive intellectual property of Nutritionix and is protected under applicable intellectual property laws and treaties. The data and resources obtained were used solely for personal and academic purposes, and were not used for commercial or revenue-generating applications. This API provided the data about the calories, proteins, fats and carbohydrates for each ingredient. These resources empower our application with the information about macro nutrients. This functionality helps with selecting proper recipes for people with different needs.

Together, joined data from both services serve as the foundation for our database. Recipes, ingredients and all information about them are crucial for further creation of all features of our system. Only then can the system deliver valuable insights and functionalities centered around recipes and their nutritional values.

### 5.2. *Database design*

WIKTOR CZETYRBOK, KAMIL DĘDZA, AGNIESZKA KULETA

To efficiently store and manage all necessary data, a relational database was selected as the most suitable solution. This decision was influenced by the relatively static nature of the schema, ensuring that the structure of the data would not require frequent or drastic modifications. The relational model's natural capability to provide a systematic and organized structure aligns well with the project's needs. Additionally, the relational database model offers ease of use, facilitating straightforward implementation, querying, and maintenance. Its compatibility with structured data and robust support underscores its suitability for this use case. Consequently, the relational database model emerged as the optimal choice to balance efficiency, and usability in managing the data.



Well designed schema serves as a base for further development of a system, especially storing the data itself. To visualize the schema, we used the dbdiagram.io [36]. The tool is simple, but provides all the necessary features. It enhances the process of creating the schema, with for example visual connections between the tables, or support for importing/exporting SQL code. The collaboration using such tools is straightforward, and developers can focus purely on the design, which is much easier when compared to brainstorming only with code.

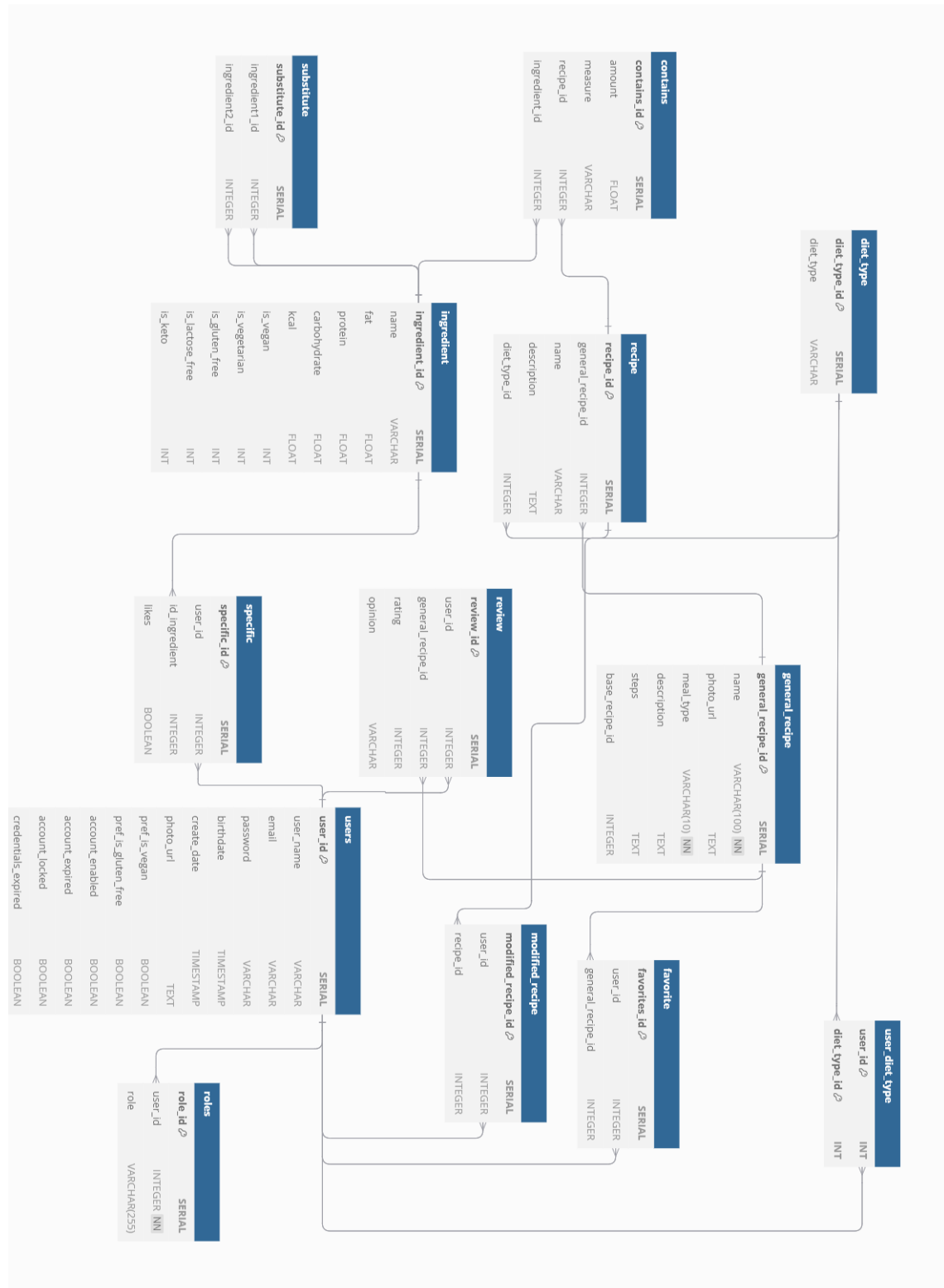


Fig. 5.1. Database schema.

The data dictionary is available at **Appendix B ??**.

### 5.3. Transformations

KAMIL DĘDZA

API requests were executed using the Python requests library [37]. To ensure security and prevent unauthorized access, the specific API keys and application IDs were stored in an environment file (.env), which was not exposed to the public or included in the source code of our repository. This approach ensures efficient and safe usage of API [35].

```
1 headers = {
2     'Content-Type': os.getenv('CONTENT_TYPE'),
3     'x-app-id': os.getenv('X_APP_ID'),
4     'x-app-key': os.getenv('X_APP_KEY')
5 }
6 body = {
7     "query": ingredient # parameter with name of particular ingredient
8 }
9 try:
10     response = requests.post(
11         'https://trackapi.nutritionix.com/v2/natural/nutrients',
12         headers=headers,
13         data=json.dumps(body)
14     ).json()
15 except requests.exceptions.RequestException as e:
16     print(f"Error while getting the data: {e}")
```

Fig. 5.2. Headers safely stored

Moving forward, after having created the target tables and finding appropriate data sources, it is time to actually load the data. The responses come with data, which need to be transformed. Then and only then are we able to make use of these resources. We have organized our structure in such a way that for each table we have created a Python script, which transforms the data and generates files which later can be loaded into the database. Whole process follows a standard, production approach called ETL meaning extraction, transformation, and loading phases, which are used to migrate heterogeneous data from one or more data sources into a target system [38].

To start with, the code snippet in Figure ?? presents a simple extraction and transformation of the source fields into more of our desired structure.

```

1 name = response.get('strMeal') # Validate 'strMeal' existence and value
2 if not name:
3     return False
4 # Extract and clean fields
5 name = name.replace("'", "")
6 instructions = (response.get('strInstructions', "").replace("'", ""))
7                 .replace("\n", " ").replace('\r', " ")
8 pho_url = response.get('strMealThumb', "").replace("'", "")

```

Fig. 5.3. Extraction of fields from JSON response

In majority of cases, this type of simple extraction is enough for our cases, meaning finding the proper field in response, then applying some cleaning function, in that case replace [39], which ensures that the input is properly formatted. What is worth mentioning, it is also safe to use this approach from Figure ?? even if there is no such field with a particular name, because then the second parameter from `get()` function is returned, in our case it would be an empty string.

For some tables it was necessary to get the primary key of other tables. One example of such usage can be the substitute table. The ingredient pairs are stored in the plain txt file as key and value pairs. So at first, we loaded the file, then we fetched the keys of ingredients from the pairs from an already existing table. If both were found, then we can save them into a file, and then load them into a table. So for this scenario, we have to keep in mind that the order of execution plays a role. Without an ingredient table we are not able to add the substitutes.

```

1 with open(self.substitutes_map, mode='r', encoding='utf-8') as file:
2     substitutes_map = csv.reader(file)
3     for lines in substitutes_map:
4         ingredient_1 = lines[0].lower().replace("'", "").replace(", ", " ")
5         ingredient_2 = lines[1].lower().replace("'", "").replace(", ", " ")
6         id_ingredient_1 = self.get_id("ingredient", "ingredient_id",
7                                     ingredient_1.replace("'", ""))
8         id_ingredient_2 = self.get_id("ingredient", "ingredient_id",
9                                     ingredient_2.replace("'", ""))
10        if id_ingredient_1 is None or id_ingredient_2 is None:
11            print(f"Missing ID for {id_ingredient_1} or {id_ingredient_2}")
12            return
13        else:
14            with open("substitutes_table.txt", "w", encoding='utf-8') as sub_file:
15                # saving to file
16                sub_file.write(f"DEFAULT,{id_ingredient_1},{id_ingredient_2}")

```

Fig. 5.4. Fetching the primary keys of existing ingredients to make substitutes

The load phase, the final stage of the ETL process, involves transferring the cleaned and transformed data into the target data repository, which in this case is a Google Cloud Platform (GCP) PostgreSQL [40] instance. To maintain the security best practices, here we have again

made use of the .env file, where we keep all our secret keys and password, without exposing them to public repositories. It also helps with future development of the code, making it easy to change for example password or user name. Our script takes the table name and path to a file which we want to load, and then after creating a query we execute it with the help of a cursor [41]. Once created, this object allows developers to execute SQL statements like select, update, insert and delete. It also enables operations such as commit and rollback, making it easier to control changes. Query uses %s placeholders and passes data as a parameter to cursor.execute(). This ensures the database handles escaping and type conversion properly, avoiding SQL injection.

```

1  PARAMS = {
2      'dbname': os.getenv('GCP_DB_NAME'),
3      'user': os.getenv('GCP_DB_USER'),
4      'password': os.getenv('GCP_DB_PASSWORD'),
5      'host': os.getenv('GCP_DB_HOST'),
6      'port': os.getenv('GCP_DB_PORT')
7  }
8  def load_file_to_database(file_name, table_name):
9      try:
10         # connect to the PostgreSQL database and create a cursor
11         connection = psycopg2.connect(**PARAMS)
12         cursor = connection.cursor()
13         # open the file and read lines
14         with open(file_name, 'r', encoding='utf-8') as file:
15             for line in file:
16                 row_data = tuple(line.strip().split(',')) # convert to tuple
17                 # construct the query with placeholders
18                 placeholders = ', '.join(['%s'] * len(row_data))
19                 query = f"INSERT INTO {table_name} VALUES ({placeholders});"
20                 try:
21                     cursor.execute(query, row_data) # pass row_data as parameters
22                 except (Exception, psycopg2.DatabaseError) as error:
23                     print(f"Error while inserting row: {error}")
24                     connection.rollback() # roll back if there's an error
25                 else:
26                     connection.commit()

```

Fig. 5.5. Connection to GCP PostgreSQL instance and execution of the query

All missing or empty values in the dataset were systematically identified and addressed. Intuitive dummy values, such as n/a(not available) for text fields were used to ensure clarity and maintain consistency in the data. To ensure that there are no anomalies in our database, we provided necessary checks and constraints. All columns that are referencing to another table, hence storing a foreign key were marked as such, so only the values existing in the table that they are pointing to are possible. For other columns, for example, name in the general\_recipe table is restricted by the NOT NULL statement, which prevents adding ingredients without a name.

Columns, where range of values is expected are also limited with proper check. Taking as an example the 'kcal' column from table 'ingredient', we know that it should only accept values from range 0 to 900.

```
1 alter table ingredient
2 alter column name set not null;
3 alter table ingredient
4 add constraint expected_macro check (fat >= 0 and carbohydrate >= 0
5 and protein >= 0 and kcal >= 0
6 and fat <= 100 and carbohydrate <= 100 and protein <= 100
7 and kcal <= 900);
8 alter table ingredient
9 add constraint valid_flags check (is_keto in (0,1) and is_vegan in (0,1)
10 and is_gluten_free in (0,1) and
11 is_vegetarian in (0,1) and is_lactose_free in (0,1));
```

Fig. 5.6. Constraints on ingredient table

To sum up, our data handling process starts with making use of existing and open APIs. Then we suit the data for our purposes, which includes cleansing and transformation. When all the information is ready to be stored in persistent storage, we load them into the PostgreSQL [40] service for databases provided by GCP. After that step, our other services can take advantage of such data, and empower our solution with valuable features and information.

## 6. SYSTEM DESIGN

### 6.1. Prototype

AGNIESZKA KULETA

Having prepared principal requirements we wanted to validate them and make sure whether we have include everything we needed to make our system work. To do it, we have chosen the technique of validation called prototyping. This simplified model helped us to detect usability errors earlier and correct them not only on lower cost, but also it was more time efficient. Our prototype was design as a Hi-Fi prototype using **Figma** environment and it can be accessed here. It is working and interactive, so it was perfect to present it to our stakeholders and demonstrate how are software would look like and which usability features it will include. Moreover, we wanted to make sure that the design of our system will ensure pleasure user-experience. To achieve that we focused on the following features:

- **Readable text**
- **A large number of images**, to enhance recognizability over text
- **Numerous icons**, that are functional and visually distinctive
- **Bright, well-matched colors**, that are engaging without being overwhelming

We have tried our portal to look simple and elegant, we did it by making it visually consistent and standardize. Moreover, we have prepared navigation like bottoms, flags and labels.

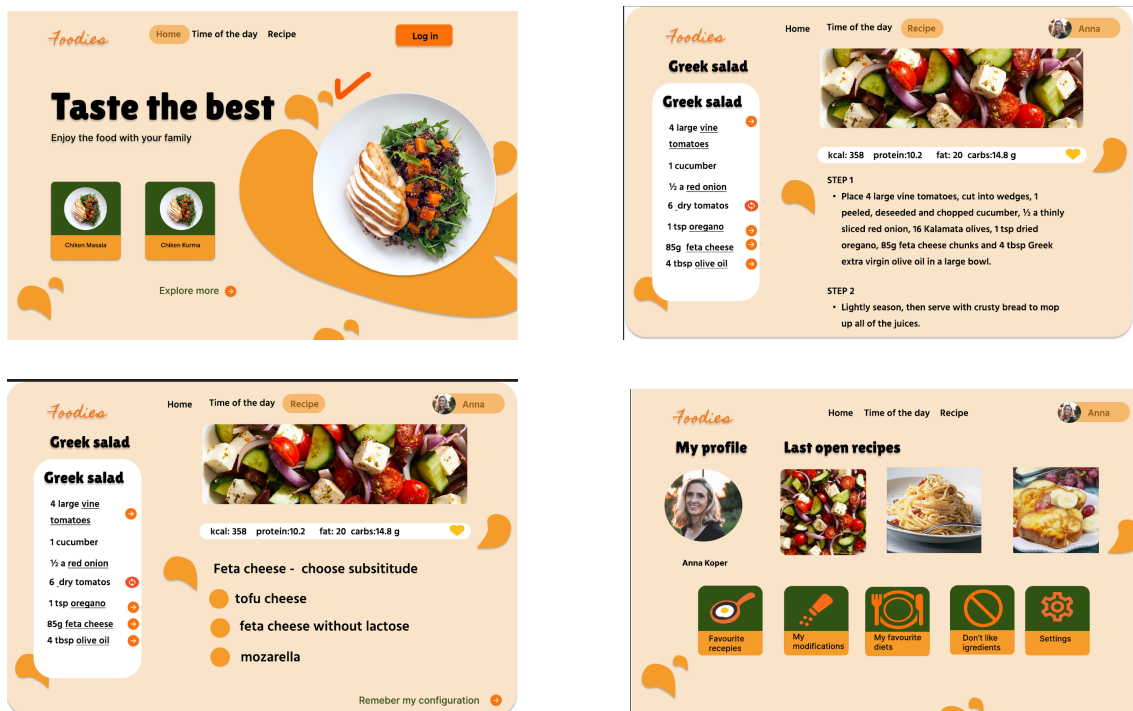


Fig. 6.1. Four prototype screens arranged in a 2x2 layout.

Finally, when we got our vision of design and all requirements verified and gathered we started to build the architecture of our system which will be described in the next section of the chapter.

## 6.2. Application Architecture

WIKTOR CZETYRBOK

### 6.2.1. Backend Service: Java Spring Boot

For the backend of our application, we decided to use Spring Boot, it is a robust framework that provides an efficient way to implement a scalable, modular architecture using the MVC (Model-View-Controller) design pattern. Spring Boot is particularly well-suited for building RESTful APIs, which are the base of communication in a microservices-based architecture.

Spring Boot simplifies the creation of RESTful web services by using its built-in support for Spring Web. REST controllers manage incoming requests, route them to the appropriate service layer, and return responses in standard formats like JSON. Figure ?? shows an example of a REST API controller from culinary-user-application (backend service), which shows how to handle common HTTP methods such as GET and PUT.

```
1  @RestController
2  @RequestMapping(path = "/api/user")
3  public class UserController {
4      private final UserService userService;
5
6      @GetMapping("/username/{userName}")
7      public UserDetailsDTO getUserByUserName(@PathVariable String userName) {
8          return userService.getUserByUserName(userName);
9      }
10
11     @PutMapping("/{userId}")
12     public UserDetailsDTO updateUser(@PathVariable long userId,
13         @RequestBody PutUserDTO putUserDTO) {
14         return userService.updateUser(userId, putUserDTO);
15     }
16 }
```

Fig. 6.2. Example of REST controller in culinary-user-service

The Spring Boot annotations in the code simplify the creation of RESTful web services. The `@RestController` annotation identifies the class as a REST controller, where the return values of the methods are automatically serialized into JSON and included in the response body. `@RequestMapping(path = "/api/user")` maps HTTP requests that match the specified path (`/api/user`) to this controller. The `@GetMapping("/username/{userName}")` annotation maps HTTP GET requests to the method, with `userName` representing a dynamic path variable passed to the method. `@PathVariable` binds the method parameter to the path variable from the URL. The

`@PutMapping("/userId")` annotation maps HTTP PUT requests to the method, with `userId` as a dynamic path variable. Finally, `@RequestBody` tells Spring to deserialize the HTTP request body into a specified Java object, in this case, `PutUserDTO`. With this standardized use of annotations, it becomes quick and easy to create a set of APIs, without the complexity of manually handling HTTP request structures.

Spring Boot implements the Model-View-Controller paradigm to organize code effectively. This pattern helps to divide code into distinct layers, each of which carries out a specific set of tasks as shown in the Figure ??).

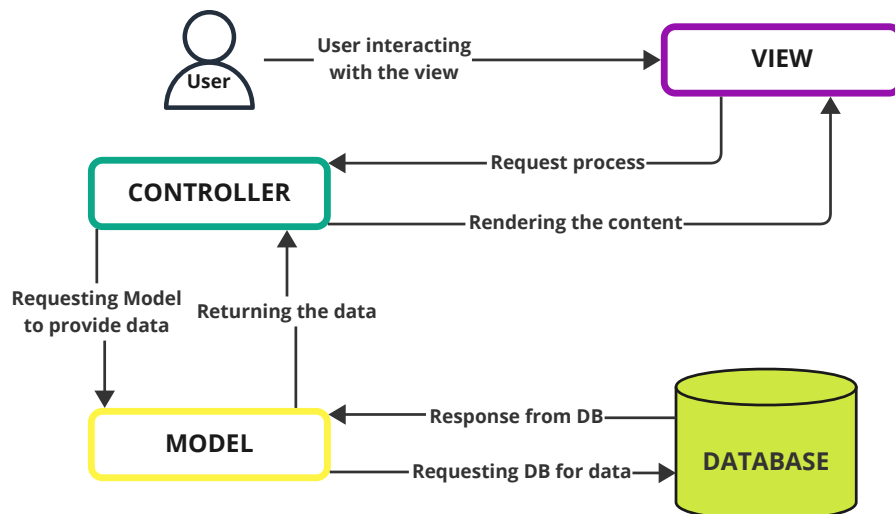


Fig. 6.3. Model View Controller design pattern visualization (created by the Authors)

While we have already described the Controller in Spring Boot, the Model and View components are crucial in maintaining this separation of concerns.

The Model in Spring Boot represents the data and the business logic of the application. It typically consists of Java classes that represent entities, such as users or orders, and contain properties, along with methods to manipulate and process the data. The Model is responsible for encapsulating the application's state and handling the logic required to access and update that state. For example, a User class might contain properties such as `userName`, `email`, and `password`, along with getter and setter methods to manage those values. The Model can also interact with the database through repositories and services to persist and retrieve data.

The View is typically represented as HTML page in web browser, in this architecture it is interpreted as the JSON responses returned by the Spring Boot backend. These JSON responses are later consumed by the Angular frontend, which renders the data in the browser, as described in chapter ??.

As the service grows, it becomes increasingly challenging to manage and document all the endpoints and DTOs (Data Transfer Objects) used in the application. This is where the Swagger tool comes in handy. Swagger, also known as OpenAPI [42], provides an intuitive interface for automatically documenting and visualizing REST APIs. Swagger helps by listing all available endpoints, their HTTP methods, request/response formats, and associated DTOs, making it easy to understand and test the API. The interactive UI (Fig. ??) enables developers



and stakeholders to explore the API, test endpoints, and ensure consistency without relying on external tools. We implemented this in our backend service, as it exposes over 30 endpoints and multiple DTOs.

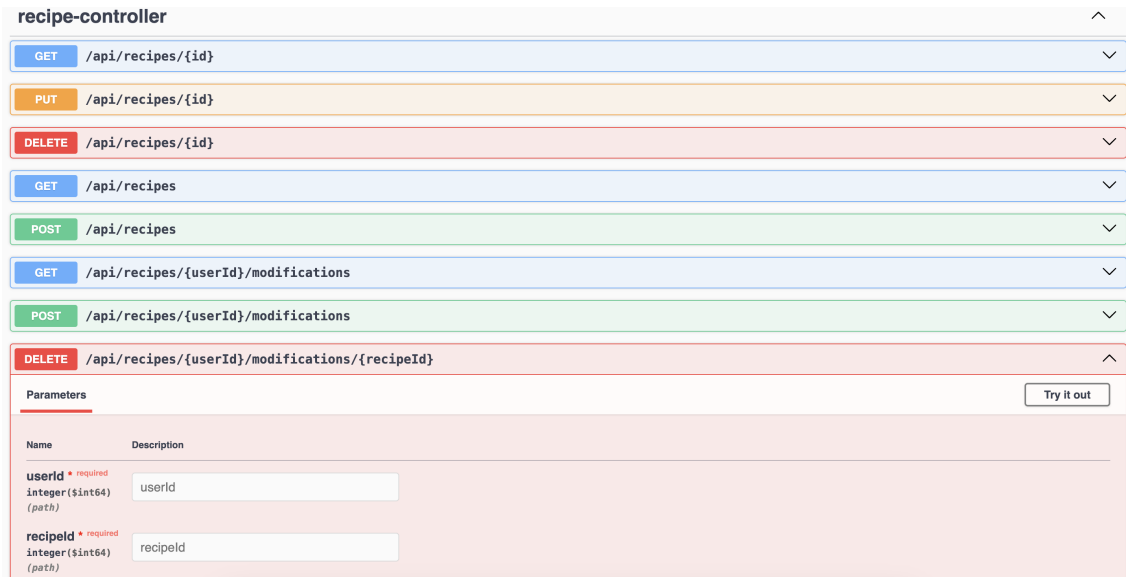


Fig. 6.4. OpenAPI Swagger interface for recipe-controller in culinary-user-service

### 6.2.2. Frontend: Angular

Angular [43] is a platform and framework for building single-page applications (SPAs) using HTML and TypeScript. In the culinary-frontend-service, Angular is used to create an interactive user interface that communicates with the backend through REST API calls. The `HttpClient` module in Angular simplifies making API requests to endpoints provided by the culinary-user-service, supporting data exchange for operations like user authentication, user management, retrieving recipes and other business data.

Angular's component-based structure ensures modular, reusable UI elements, which makes the application scalable and maintainable as it grows. Additionally, Angular's `RxJS` (Reactive extensions library for JavaScript) [44] integration efficiently manages asynchronous API responses and application state, making the frontend responsive.

Moreover, one of the key advantages of using Angular is its two-way data binding. This feature keeps the UI in sync with updates from the backend in real time, resulting in a dynamic and engaging user experience. As shown in Figure ??, any change made in the view is immediately reflected in the model, and any update in the model is sent to the view. Since the view is simply a representation of the model, the controller is separated from the view.

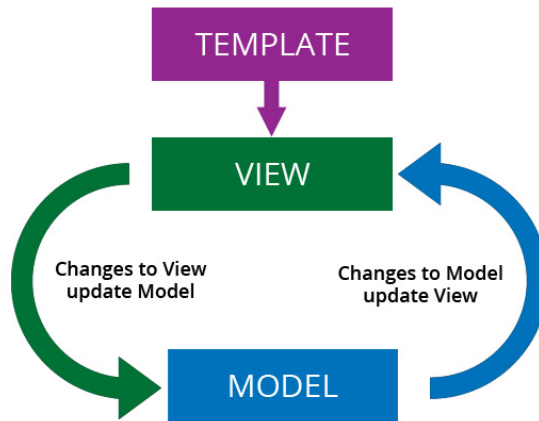


Fig. 6.5. Two-way data binding [source: [stackoverflow.com/questions/13504906/what-is-two-way-binding](https://stackoverflow.com/questions/13504906/what-is-two-way-binding)]

To improve the design and styling of the frontend, we decided to use Tailwind CSS. Tailwind's utility-first approach which means that it allows for highly customizable, responsive layouts without the need for writing custom CSS for every component. This helps speed up the development process, ensuring that the frontend not only looks polished but also remains flexible and adaptable to future changes. Tailwind's classes can be easily applied to components, allowing for quick adjustments to the UI with minimal effort, while maintaining a consistent design system across the application.

### 6.2.3. Data Layer with Java Persistence API: PostgreSQL

Java Persistence API (JPA) [45] is an Object-Relational Mapping (ORM) tool for the data layer in our application, enabling interaction with a relational database like PostgreSQL. JPA abstracts database operations by mapping Java objects to database tables, eliminating the need to write boilerplate SQL code. This simplifies data management, ensures consistency, and increases developer productivity.

During local development, we also use H2 DB, it is an in-memory database that integrates with JPA. H2 allows rapid prototyping and testing without the overhead of setting up all the environment that comes when creating new database. Because we use JPA mappings switching between H2 and PostgreSQL is very straightforward, without any code changes it allows to test the app logic, achieved by only changing the database configuration in the application properties. This setup mirrors the structure of the PostgreSQL database, ensuring that development environments are closely aligned with the production.

In JPA, entities are Java classes that represent database tables Figure ??.

```

1  @Table(name = "users") // specifies SQL table name
2  @Entity // Maps class to a database table
3  public class User implements Serializable {
4      @Id // marks as primary key
5      @GeneratedValue(strategy = GenerationType.IDENTITY) // auto-generates id
6      @Column(name = "user_id") // maps to specific column
7      private Long id;
8      @Column(name = "user_name")
9      private String userName;
10     @Column(nullable = false) // ensures that field is not null
11     private String password;
12 }

```

Fig. 6.6. Example of JPA Table

JPA simplifies relationships between entities, such as a one-to-many association between a User and Role, by using specific annotations, as shown on Figure ??.

```

1  @OneToMany(mappedBy = "user")
2  private Set<Role> roles = new HashSet<>();

```

Fig. 6.7. One-to-many JPA mapping

After configuring all the relationships and mappings for our database entities, we can use Spring Data JPA to interact with the database through repository interfaces. By extending JpaRepository interface, we gain access to a set of predefined methods for common operations such as saving, updating, and retrieving records, without needing to implement them ourselves. This eliminates the need to write boilerplate code for queries. JPA allows us to define custom queries using a syntax similar to SQL. Figure ?? demonstrates how to create a repository for the User entity, including a custom query to search for usernames (line 3-5) using a regular expression and predefined query to find a user by email. The findByEmail (line 7) method is automatically implemented by JPA, as long as the naming convention is followed and the correct parameter is provided.

```

1  public interface UserRepository extends JpaRepository<User, Long> {
2
3      @Query("SELECT c FROM User c WHERE LOWER(c.userName)" +
4            " LIKE LOWER(CONCAT('%', :userName, '%'))")
5      Optional<User> findByUserNameRegex(@Param("userName") String userName);
6
7      Optional<User> findByEmail(String email);
8  }

```

Fig. 6.8. JPA repository interface

#### 6.2.4. User session management: Redis

Redis [46] is an in-memory key-value database that is widely used for caching or session management. Making it an excellent choice for storing user sessions in the culinary-user-service application. By leveraging Redis, the application benefits from enhanced performance, scalability, and reliability. Performance is a key advantage of Redis, as it stores data in memory, enabling extremely fast read and write operations, which is essential for session management. Redis is also highly scalable, capable of handling a large number of read and write operations per second, making it ideal for high-traffic applications. Furthermore, Redis can be scaled horizontally by adding additional nodes to the cluster, supporting the growth of the application. Another significant feature of Redis is its support for data persistence, which allows session data to be saved to disk and restored in the event of a server restart, ensuring that user sessions are not lost during a failure. Additionally, Redis integrates well with Spring Boot, simplifying its setup and use for session management. Spring Session, for example, provides robust support for managing user sessions in a distributed environment using Redis, enabling consistent and reliable session handling across multiple instances of the application.

Redis was configured in backend application by using *spring-boot-starter-data-redis* package. Figure ?? illustrates the configuration class used to establish the Redis connection and enable session database integration within the backend application.

```
1  @Configuration
2  // Enables Redis for managing Spring session data.
3  @EnableRedisIndexedHttpSession
4  @RequiredArgsConstructor
5  // Activates this configuration only in non-local and non-test profiles.
6  @Profile({"!local & !test"})
7  public class RedisConfig extends AbstractHttpSessionApplicationInitializer {
8
9      // Connection properties, injected from the application's configuration file
10     private final RedisProperties redisProperties;
11
12     @Bean
13     public LettuceConnectionFactory connectionFactory() {
14         RedisStandaloneConfiguration configuration =
15             new RedisStandaloneConfiguration(this.redisProperties.getHost(),
16                 this.redisProperties.getPort());
17         configuration.setPassword(redisProperties.getPassword());
18         // Returns a factory for creating connections to Redis.
19         return new LettuceConnectionFactory(configuration);
20     }
21
22 }
```

Fig. 6.9. Configuration class for Redis connections and sessions

Configuration for Redis-Backed session management in a Spring properties is defined in application.yaml file, Figure ??).

```
1 server:
2   servlet:
3     session:
4       cookie:
5         domain: localhost # session cookie domain for local development.
6         http-only: true # not accessible via JavaScript
7         max-age: 600 # max age of the session cookie
8         name: JSESSIONID # cookie name to track the session.
9         path: / # path where cookie is valid, set to the root
10        # blocks the browser from sending the cookie with cross-site requests
11        same-site: strict
12        timeout: 1m # session timeout duration
13        tracking-modes: cookie # cookies are used to track the session
```

Fig. 6.10. Session management configuration properties

We set up configuration classes for local Redis connection and session database integration in development and test environments. In local/test versions we create Redis instance in applications runtime, as shown on Figure ??.

```
1 @Configuration
2 @Profile({"local", "test"})
3 public class RedisConfig {
4
5     private final RedisServer redisServer;
6
7     public RedisConfig() throws IOException {
8         this.redisServer = new RedisServer(6379);
9     }
10
11     @PostConstruct
12     public void postConstruct() throws IOException {
13         redisServer.start();
14     }
15
16     @PreDestroy
17     public void preDestroy() throws IOException {
18         redisServer.stop();
19     }
20 }
```

Fig. 6.11. In memory Redis database configuration

After runtime database is created, we can connect to it and enable Redis repositories by providing DB host as localhost and appropriate port defined before, as can be seen in Figure ??.

```
1  @Configuration
2  @EnableRedisRepositories
3  @Profile({"local", "test"})
4  public class LocalBeanConfig {
5
6      @Bean
7      public LettuceConnectionFactory redisConnectionFactory() {
8          return new LettuceConnectionFactory("localhost", 6379);
9      }
10
11     @Bean
12     public RedisTemplate<?, ?> redisTemplate(
13         LettuceConnectionFactory connectionFactory) {
14         RedisTemplate<byte[], byte[]> template = new RedisTemplate<>();
15         template.setConnectionFactory(connectionFactory);
16         return template;
17     }
18 }
```

Fig. 6.12. Enabling Redis Repositories

#### 6.2.5. Containerization: Docker

Containerization [47] is a software deployment technology that packages an application and all its dependencies (e.g., libraries, configuration files) into a single lightweight, portable unit called a container. These containers can run consistently across different computing environments, such as development and production. When first introduced, containerization was revolutionary because it allowed engineers to streamline the development process by eliminating the need to set up complex dependencies, libraries, environments, and manage virtual machine images.

To implement it in our services, we decided to use Docker platform. Docker is a platform that allows you to "build, ship, and run any app, anywhere." It has come a long way in an incredibly short time and is now considered a standard way of solving one of the most expensive aspects of software: deployment, as noted by Ian Miell and Aidan Hobson S. [48].

In our case, containerization also streamlined the deployment process by creating a local development environment that closely mimics the cloud environment. Using Docker, we hosted essential services such as databases in containers, ensuring consistency across environments. By using docker-compose, we could define the multi-container setups with ease.

Figure ?? demonstrates how instances for PostgreSQL and Redis are configured with Docker in our local environment, mimicking the cloud based one. Specific ports have to be exposed, and environment variables are set to manage database credentials and other settings.

With Docker running in the background, executing the command `docker-compose up` initializes the services automatically, downloading the required images from an online repository if not already available locally.

```
1 version: '3.9'
2 services:
3   db:
4     image: postgres:latest
5     ports:
6       - "5432:5432"
7     restart: always
8     environment:
9       POSTGRES_DB: culinary
10      POSTGRES_USER: postgres
11      POSTGRES_PASSWORD: postgres
12
13   redis:
14     image: redis:alpine
15     container_name: redis
16     ports:
17       - "6379:6379"
18     environment:
19       REDIS_HOST: localhost
20       REDIS_PASSWORD: session
```

Fig. 6.13. Docker compose file for setting up the databases

The cloud hosting engine discussed in Section ?? uses images specified by Dockerfiles for both the backend and frontend services. Dockerfiles are essentially sets of instructions for creating an image of the application. For the Angular frontend service, we have defined the Dockerfile, displayed on Figure ??.

```
1 FROM node:14 AS build
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 COPY . .
6 RUN npm run build
7
8 FROM nginx:alpine
9 COPY --from=build /app/dist/culinary-frontend-service /usr/share/nginx/html
10 COPY nginx.conf /etc/nginx/conf.d/default.conf
11 EXPOSE 8080
12 CMD ["nginx", "-g", "daemon off;"]
```

Fig. 6.14. Dockerfile for Angular frontend service

In this Dockerfile, we first specify a base image using `FROM node:14 AS build`, which leverages Node.js version 14 for the build process. The working directory is then set to `WORKDIR /app`, where next operations will be done. The command `COPY package*.json ./` ensures that the `package.json` and `package-lock.json` files, which contain all needed libraries, are copied to the container for dependency management. Next, the command `RUN npm install` installs all necessary dependencies, followed by `COPY . .`, which copies the remaining project files into the container. The build process is then executed with `RUN npm run build`, producing the production-ready files in the `dist/` directory.

After the building stage, we switch to the Nginx [49] base image by using `FROM nginx:alpine`, a popular, lightweight, and optimized server for serving files. The command `COPY --from=build /app/dist/culinary-frontend-service /usr/share/nginx/html` moves the built files from the first stage to Nginx's default HTML directory. We then include a custom Nginx configuration using `COPY nginx.conf /etc/nginx/conf.d/default.conf`, where the Nginx properties are set up. The command `EXPOSE 8080` declares that the container will listen for requests on port 8080, which is the default port used by Google App Engine Flex (section ??). Finally, the command `CMD["nginx","-g","daemon off;"]` starts the Nginx server, allowing the container to stay active.

### 6.3. Cloud Infrastructure and Deployment

WIKTOR CZETYRBOK

This chapter outlines the components of the cloud infrastructure used in this project and details the deployment process. It describes the integration of various Google Cloud services and explains how they facilitate the development, deployment, and scaling of the application. To deploy services, establish secure and efficient communication between components, and connect to the database, the project leverages multiple cloud services. These services, hosted by third-party providers, function as infrastructure, platforms, or software that support the developers.

#### 6.3.1. Google App Engine Flex

Google App Engine (GAE) Flexible Environment [50] is an excellent platform for deploying modern, containerized applications like `culinary-user-service` (backend in Spring) and `culinary-frontend-service` (frontend). Unlike the standard environment, GAE Flex offers greater flexibility by supporting Docker-based deployment, allowing developers to define custom runtime environments tailored to their application requirements. Setting up and managing services on GAE Flex is straightforward, thanks to its intuitive configuration process and minimal operational overhead. Deployment can be accomplished with just a few commands. This makes it suitable for microservices architectures as in case we are experiencing with our application.

One of the standout features of GAE Flex is its automatic scaling capability, which can adjust the number of instances based on real-time traffic and resource usage. This ensures that applications can handle varying loads efficiently without requiring manual intervention. For example, if the number of requests to the `culinary-user-service` increases significantly during peak hours, GAE Flex will spin up additional instances to meet the demand, maintaining performance



and minimizing latency. It works also in contrary case, during off-peak times, it scales down to reduce costs. The resources that are going to be utilized in application and auto-scaling behavior is controlled by configuration values defined in the app.yaml file, shown in Figure ??.

```

1 runtime: custom # custom allows to define a Dockerfile
2 env: flex
3 service: culinary-user-service
4 automatic_scaling:
5   min_num_instances: 1 # min instances to always keep running
6   max_num_instances: 10 # max instances to scale up
7   # specifies the CPU utilization level at which new instances are added.
8   target_cpu_utilization: 0.75
9 resources: # specifies resource allocation for each instance
10  cpu: 4 # number of CPUs allocated
11  memory_gb: 5
12  disk_size_gb: 10

```

Fig. 6.15. app.yaml file for GAE configuration

GAE also has advantage in providing a comprehensive monitoring interface through the Google Cloud Console. Developers can track resource utilization, such as CPU usage, memory consumption, virtual machine traffic and instance counts and more in real time. The user-friendly dashboards (Figure ??) make it easy to identify bottlenecks, optimize application performance, and manage costs. Logs from both culinary-user-service and culinary-frontend-service are aggregated in a centralized location, simplifying debugging and providing valuable insights into the application's behavior. These logs are accessible via the Logs Explorer, which features a user-friendly interface and a query language, making it easy to locate specific information and troubleshoot issues efficiently.

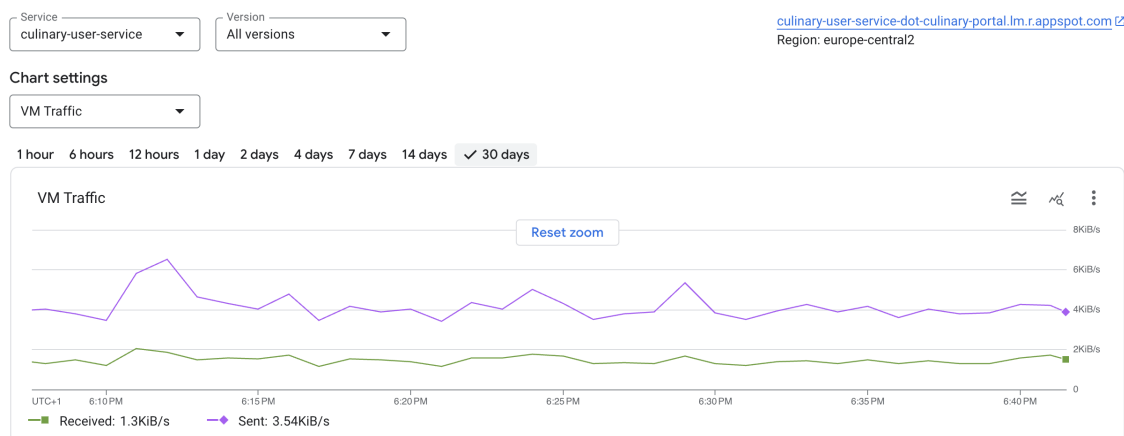


Fig. 6.16. GAE monitoring dashboards (culinary-user-service).

### 6.3.2. Memorystore: Redis

Memorystore [51] is a Google Cloud-managed Redis service, serves as the cloud-based version of the Redis database in our application architecture. Being a fully managed service

meaning developer does not have to set up any machines, the management, patching, backup. Memorystore is easy to set up and maintain, eliminating much of the complexity typically associated with self-hosted Redis deployments. As a cloud-native solution, it uses Google Cloud's infrastructure to deliver high-speed, low-latency data retrieval, making it an ideal choice for tasks such as caching and session management, which is crucial for us. One of the key advantages of using Memorystore is its high availability, ensuring that data is accessible even in the event of failures or maintenance. Furthermore, it provides exceptional scalability, enabling the system to grow alongside the application's needs. As the application scales, we do not have to worry about data migration or managing additional infrastructure. Memorystore allows us to simply scale up or out, ensuring continuous performance improvements without significant changes to the architecture. It also integrates well with the Spring framework through the Spring Data Redis library and Cloud GCP library, allowing for straightforward configuration and management and session handling within the application

### 6.3.3. Cloud SQL: PostgreSQL

Cloud SQL [40] is a fully managed relational database service, is used in our application to host PostgreSQL, providing a reliable and scalable solution for managing structured data. Like Memorystore ??, Cloud SQL eliminates the complexity of database administration, offering automated backups, patching, and seamless scaling, which allows developers to focus on building features rather than maintaining infrastructure. The service is designed to deliver high availability and resilience, ensuring that data is accessible even in the event of failures or maintenance. Cloud SQL provides seamless integration with the Google Cloud ecosystem, ensuring that our PostgreSQL instance works efficiently with other services like Google App Engine. This integration ensures that as our application grows, we can scale the database with minimal effort and without worrying about migration or changing the system. The service also integrates well with the Spring framework, allowing developers to manage PostgreSQL connections through Spring Data JPA by adding connection details in application properties, as shown in Figure ??.

```
1  spring:
2    datasource:
3      username: ${DB_USERNAME}
4      password: ${DB_PASSWORD}
5      driver-class-name: org.postgresql.Driver
6  cloud:
7    gcp:
8      sql:
9        enabled: true
10       database-name: ${DB_NAME}
11       instance-connection-name: ${DB_CONNECTION_NAME}
12  credentials:
13    encoded-key: ${GCP_SA_JSON} # for authenticating to GCP resource
```

Fig. 6.17. application.yaml for Cloud SQL connection

### 6.3.4. Google Artifact Registry

Google Artifact Registry (GAR) [52] is a fully managed service for storing and managing containerized application artifacts, such as Docker images. In our project, GAR is used to securely store Docker images for both the backend and frontend services in different folders shown in Figure ??, ensuring that application artifacts are available for deployment. GAR integrates well with GAE Flex, simplifying the deployment process. It also supports versioning, making updates and rollbacks easier. To optimize costs, we have implemented a policy to remove old, unused images, reducing storage expenses while maintaining efficient artifact management within the Google Cloud environment.

| <input type="checkbox"/> | Name                                       | Format                                                                                   | Type     | Location                 | Scanning (Disabled)  | Description                                                                                                            |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|----------|--------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> |  <a href="#">culinary-frontend-service</a> |  Docker | Standard | europa-central2 (Warsaw) | Disabled                                                                                                | Docker images for frontend service  |
| <input type="checkbox"/> |  <a href="#">culinary-user-service</a>     |  Docker | Standard | europa-central2 (Warsaw) | Disabled                                                                                                | Docker...                           |

Fig. 6.18. GAR repository for docker images.

### 6.3.5. Google Service Accounts and Permissions

In a cloud environment, managing permissions and securing resources are critical aspects of application development and deployment. In the GCP project, we utilize Service Accounts combined with RBAC (Role-Based Access Control) to ensure secure and efficient access management across services and resources. Service accounts are special Google Cloud accounts designed for application and service interactions rather than human users. They play a central role in maintaining the principle of least privilege, ensuring each service or process has access only to the resources it needs.

When creating our solution, we used special accounts for separate functions aiming at reducing the potential threats in case of a security breach. All service accounts are scoped into specific RBAC roles and capabilities, which are necessary to be safe as much as possible in their operations. This organized strategy of levels of access, as well as access to resources, creates a stable environment for the more sensitive parts of the system.

For deployment purposes, we use a dedicated service account with permissions tailored for pushing Docker images to GAR and deploying applications to GAE. This account has roles like Artifact Registry Writer and App Engine Deployer assigned, ensuring that the CI/CD (Continuous Integration and Continuous Deployment) pipeline ?? can build, store, and deploy application containers securely without granting unnecessary access to other resources. The use of RBAC here helps limit what this service account can do, reducing the risk of accidental or malicious actions.

On the application side, the culinary-user-service (backend) uses a different service account to interact with Google Memorystore and Cloud SQL. This account is granted roles like Cloud Memorystore Redis User and Cloud SQL Client to enable secure access to these resources. For development purposes, as developers, we have extensive access to the resources

needed for efficient development and testing. However, full administrative control of the project is restricted to a single cloud Project Owner which is Kamil Dęda. This individual is the only one with comprehensive permissions, ensuring strict control over sensitive operations like project-wide configuration, billing, and roles management. This setup balances effective development flexibility with good security and management practices.

#### 6.3.6. *Deployment with GitHub Actions*

In modern software development, automation plays a crucial role in development processes and improving efficiency. CI/CD are essential practices for automating the building, testing, and deployment of applications. One effective way to implement CI/CD is through the use of GitHub Actions, a known tool that allows for the automation of workflows directly within GitHub repositories.

The integration of GitHub Actions with Google Cloud services facilitates the deployment of applications to GAE using Docker containers stored in GAR. This automation ensures that code changes are consistently tested, built, and deployed to the cloud environment without manual intervention of the developer, reducing space for human error and increasing the speed of project deliveries.

The workflow involves several key stages, including authenticating to Google Cloud, building and pushing Docker images, injecting necessary secrets into configuration files, and finally deploying the application to GAE. By leveraging GitHub Actions, developers can maintain an efficient and reliable CI/CD pipeline, ensuring that new features and updates are delivered seamlessly to the production environment. Workflow file for backend service is available on our github.

Figure ?? provides a visual representation of the CI/CD pipeline process with GitHub Actions, outlining each step involved from code push to deployment.

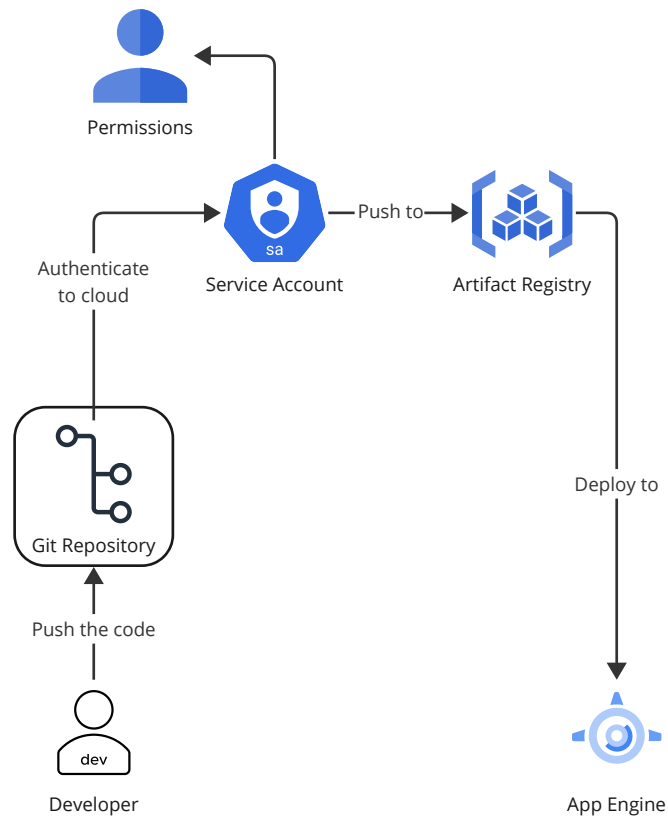


Fig. 6.19. CI/CD with Github Actions (created by the Authors)

### 6.3.7. Cloud Infrastructure Overview

The system is designed for efficient communication between these services. The culinary-user-service (backend) communicates with Memorystore Redis for user session management and Cloud SQL Postgres for accessing business data. The frontend and backend services are integrated through APIs securely, allowing the frontend to fetch data and perform operations by interacting with the backend. This clear separation of logic between the frontend and backend makes the system more maintainable, and it allows both components to scale independently depending on usage demands. The overview of communication is presented on Figure ??.

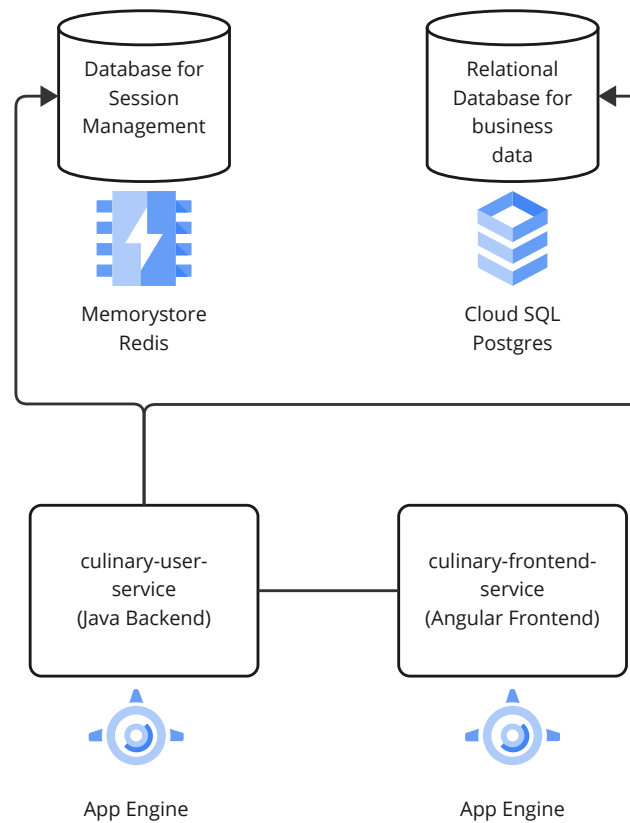


Fig. 6.20. Current cloud infrastructure (created by the Authors)

The infrastructure is very scalable and resilient, using the capabilities for automatic scaling of Google App Engine to handle different loads, reducing hosting costs. Moreover, managed services like Memorystore (Redis) and Cloud SQL provide high availability and failovers, which ensure that the data is kept securely and remains available even in outages. This architecture is a good example of various cloud engineering practices such as microservices architecture, cloud-native design, and the use of managed services to reduce the complexity of the system.

## 7. TESTING (EVALUATION, VERIFICATION)

### 7.1. *Unit testing*

WIKTOR CZETYRBOK

Without a doubt, testing is one of the essential elements in the software development life cycle. It is one of the most resource-consuming and critical stages in the application development process. A test the proper way saves you money and anguish right now, it is assuring reliability, stability, and usability all of which are fundamental for quality overall performance software. This allows you to notice and correct risks at a time you can afford to in the development cycle, which translates into lower costs and fewer failures down the line. Testing from different perspectives ensures that the software performs as intended under varying conditions, meets user requirements and validates security and performance standards.

The unit tests that we used internally to test our application, these were created in parallel with the building of new system capabilities. We used the JUnit[53] and Mockito[54] libraries to create our tests. JUnit makes available a variety of assertion methods allowing to check whether the result of executing a given set of calls will be what we are expecting. A method name like `assertNotNull` (line 15, Figure ??) serves as an example of this, which asserts that object is not null. Also with `assertThrows`, it is possible to verify that the right exceptions are thrown (in case of invalid data). Handling exception example can be found in the line 24 in Figure ??.

A great complement to the JUnit library is Mockito, a popular library that helps developers create mock objects. This is particularly useful for writing tests that isolate methods from the rest of the application, ensuring they are independent and don't rely on external components. To create mock objects, you annotate the fields with `@Mock`. Then, by calling `MockitoAnnotations.initMocks(this)`, Mockito automatically initializes these fields with mock objects. To define how these mock objects should behave during tests, you can use Mockito's `when` and `thenReturn` methods, which specify the expected responses when certain actions are performed. This approach allows for controlled and focused unit testing, as presented in lines 13 and 22-23 in Figure ??.

```

1 class AuthServiceTest {
2     @Mock private PasswordEncoder passEncoder;
3     @Mock private UserRepository userRepository;
4     @InjectMocks private AuthService authService;
5     @BeforeEach
6     void setUp() {MockitoAnnotations.initMocks(this);}
7
8     @Test
9     void testRegister_NewCustomer_Success() {
10         RegisterDTO dto = new RegisterDTO("usern", "test@ex.com", "pass");
11         when(userRepository.findByEmail(dto.email())).thenReturn(empty());
12         String encodedPassword = "encodedPassword";
13         when(passEncoder.encode(dto.password())).thenReturn(encodedPassword);
14         UserDetailsDTO result = authService.register(dto);
15         assertNotNull(result);
16         verify(userRepository, times(1)).save(any());
17     }
18
19     @Test
20     void testRegister_ExistingCustomer_ExceptionThrown() {
21         RegisterDTO dto = new RegisterDTO("user", "test@ex.com", "pass");
22         when(userRepository.findByEmail(dto.email()))
23             .thenReturn(Optional.of(new User()));
24         assertThrows(UserAlreadyExistsException.class,
25             () -> authService.register(dto));
26         verify(userRepository, never()).save(any());
27     }
28 }

```

Fig. 7.1. Unit tests scenarios for AuthService class.

For unit testing, achieving high test coverage is crucial to ensure the reliability and maintainability of the repository. Test coverage measures the extent to which the code is exercised by the unit tests, helping identify untested or poorly tested areas. To evaluate test coverage in our Java backend service, we utilize JaCoCo[55] (Java Code Coverage), a widely used code coverage analysis tool. JaCoCo provides detailed insights into which parts of the code are being tested. By integrating JaCoCo into our CI/CD pipeline, we ensure consistent tracking of test coverage over time and enforce quality standards during development. With over 120 test cases, in our application we managed to achieve overall coverage over 80%, which meets modern days business standards. Bar charts visible on Figure ?? and percentages display the coverage of specific java packages.



## culinary-user-service

| Element                                                        | Missed Instructions                                                                | Cov. |
|----------------------------------------------------------------|------------------------------------------------------------------------------------|------|
| <a href="#">com.culinary.userservice.user.enumeration</a>      |   | 100% |
| <a href="#">com.culinary.userservice.user.controller</a>       |   | 100% |
| <a href="#">com.culinary.userservice.recipe.controller</a>     |  | 100% |
| <a href="#">com.culinary.userservice.ingredient.controller</a> |   | 89%  |
| <a href="#">com.culinary.userservice.security.controller</a>   |   | 68%  |
| <a href="#">com.culinary.userservice.user.service</a>          |  | 89%  |
| <a href="#">com.culinary.userservice.security.security</a>     |   | 60%  |
| <a href="#">com.culinary.userservice.recipe.util</a>           |   | 87%  |
| <a href="#">com.culinary.userservice.security.service</a>      |  | 59%  |
| <a href="#">com.culinary.userservice.recipe.service</a>        |  | 76%  |
| Total                                                          | 430 of 2,297                                                                       | 81%  |

Fig. 7.2. Jacoco test report of culinary-user-service

### 7.2. Test scenario

KAMIL DĘDZA

There are many different tools and techniques when it comes to testing the software. One of them is a test scenario. What is it and why is it useful? The scenario test has five key characteristics, it is (a) a story that is (b) motivating, (c) credible, (d) complex, and (e) easy to evaluate [56]. For our purpose, we have created a few tasks for example users to carry out. Those are simple short stories, which are likely to occur in our system. Some of the scenarios require preconditions, meaning the expected state of the before. The scenarios contain a few simple steps. Then, after performing the steps, we can compare the predicted results with the real results that our system generated during the interaction with the user. By this assessment, the developers can see whether the solution that they have created offers required features, and that the system works well under real-life load. All scenarios were successfully completed.

### 7.3. Test Scenarios for the "Foodies" Application

#### 7.3.1. User Login Functionality

Objective:

To ensure users can log in successfully with valid credentials and receive appropriate feedback for invalid login attempts.

Actors Involved:

- End User: Anna Koper (example user)
- System: Foodies application

Preconditions:

- The user has an active account on the Foodies platform.

- The application is available and accessible.
- The user's account credentials (email and password) are known and valid.

Test Steps:

1. Access the Login Page: Navigate to the Foodies application login page. Verify that the login form is visible, with fields for email and password, and a "Login" button.

2. Enter Valid Credentials: Input the correct email address in the "Email" field (e.g., `aniakoper@gmail.com`). Input the correct password in the "Password" field (e.g., `SecurePass123`).

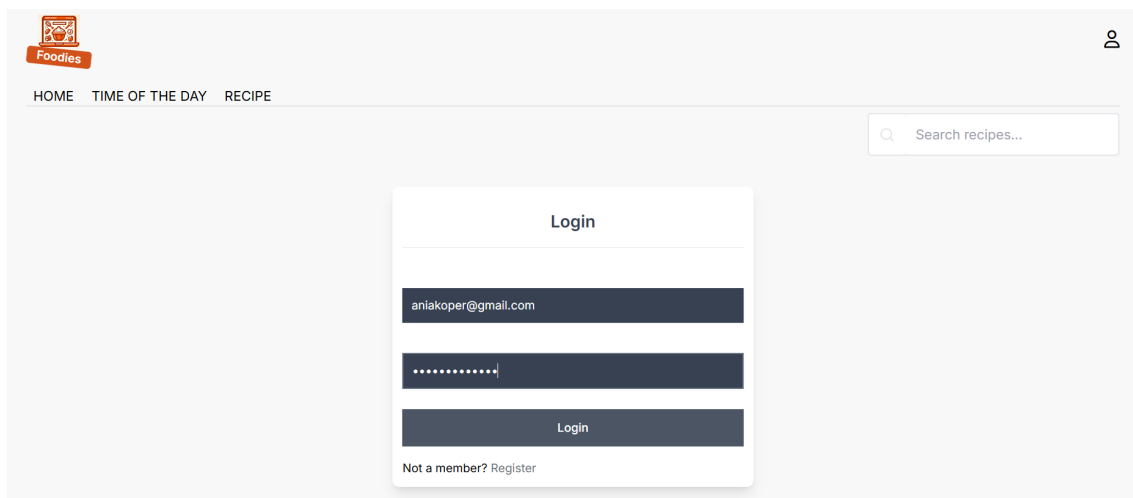


Fig. 7.3. Login Page

3. Submit Login Form: Click on the "Login" button and wait for the system to process the request.

4. Validate Successful Login: Verify that the user is redirected to the home page. Check for the availability of user-specific data (e.g., favourite recipes, profile, modifications, settings).

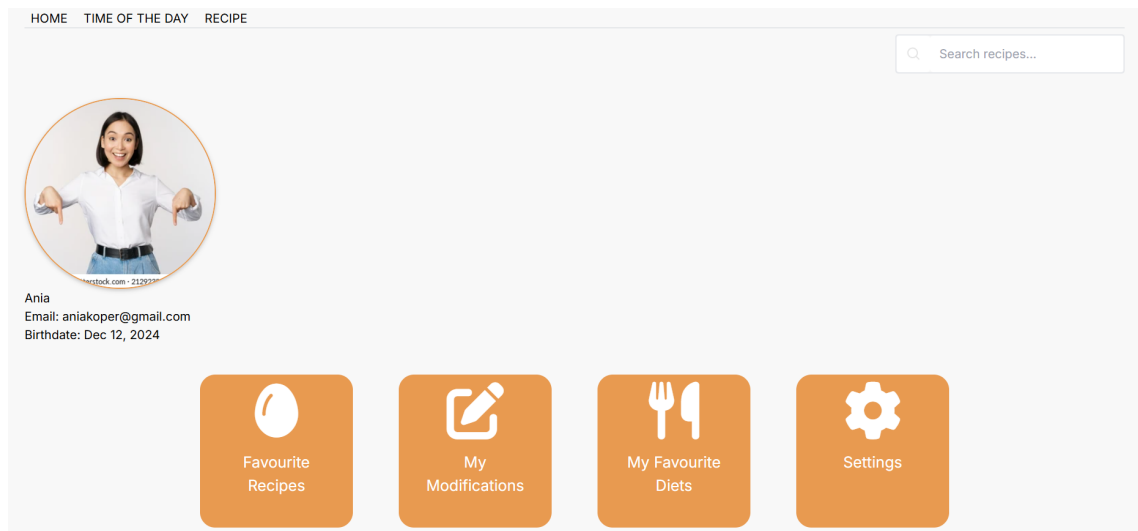


Fig. 7.4. User profile

5. Test Error Feedback (Optional Negative Scenarios): Enter invalid email or password and click "Login." Verify appropriate error messages are displayed.

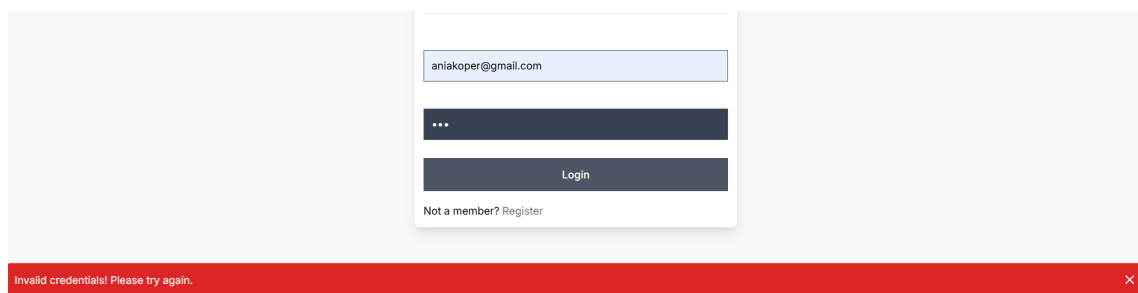


Fig. 7.5. Invalid Credentials

6. Session Validation: Ensure that the user remains logged in across the session unless they explicitly log out.

Expected Results:

- Users with valid credentials can access their personalized profile.
- Users with invalid credentials receive error messages.
- The system prevents unauthorized access to user accounts.

### 7.3.2. *Sorting and Filtering Recipes*

Objective:

To verify that users can effectively filter and sort recipes based on various parameters, such as diet, ingredients, calories, and ranking, and that the results are displayed correctly.

Actors Involved:

- End User: Anna Koper (example user)
- System: Foodies application

#### Preconditions:

- The user is logged into the Foodies application.
- The recipe database contains a variety of recipes with attributes such as dietary labels, calories, and rankings.

#### Test Steps:

1. Filtering Recipes: Navigate to the "Recipe" section and verify that the filtering options are visible.
  - Apply a single filter (e.g., Vegetarian) and verify that only recipes tagged as "Vegetarian" are displayed.
  - Apply multiple filters such as: - Calories: "Less than 1000 kcal."  
Verify that the displayed recipes satisfy all selected criteria.

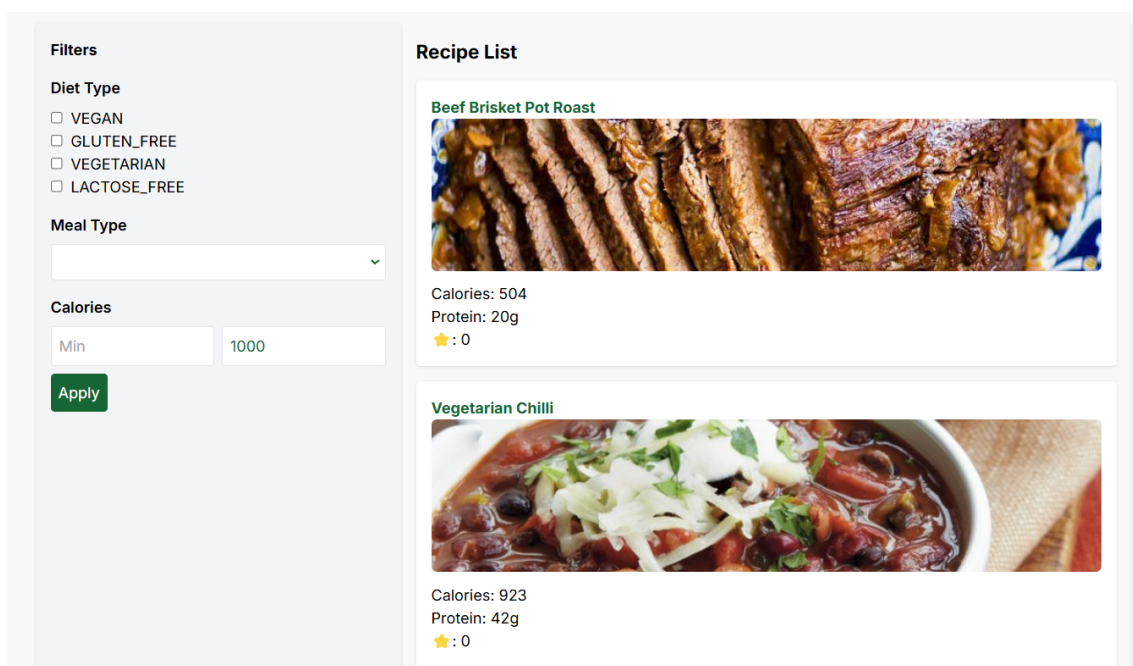


Fig. 7.6. Calories filter

#### 2. Sorting Recipes:

- Sort recipes by ranking (e.g., Ranking (High to Low)) and verify the results.
- Sort recipes by protein content (e.g., Protein (High to Low)) and verify the results.

### Recipe List

Rappie Pie



Calories: 7817  
Protein: 661g  
★ : 0

Irish stew



Calories: 9237  
Protein: 645g  
★ : 0

Fig. 7.7. Sorted recipes - protein high to low

3. Combine Sorting and Filtering: Apply a filter (e.g., Vegan) and sort the filtered results by Protein (High to Low). Verify that the recipes meet both filter and sort criteria.

Filters

Diet Type

☒ VEGAN  
☐ GLUTEN\_FREE  
☐ VEGETARIAN  
☐ LACTOSE\_FREE

Meal Type

Calories

Min  Max

Apply

Recipe List

Chivito uruguayo



Calories: 1062  
Protein: 85g  
★ : 4

Boxty Breakfast



Calories: 2277  
Protein: 78g  
★ : 0

Sort by

Calories

Calories - Low to High  
Calories - High to Low

Protein

Protein - Low to High  
Protein - High to Low

Ranking

Ranking - Low to High  
Ranking - High to Low

Fig. 7.8. Combined filters

Expected Results:

- Filtering criteria are applied accurately, and displayed recipes meet all selected conditions.

- Sorting options rearrange recipes based on the chosen attribute.
- Combining sorting and filtering works seamlessly.

### 7.3.3. Substituting Ingredients in Recipes

#### Objective:

To ensure users can substitute ingredients in recipes and verify that substitutions are accurately reflected in the recipe details.

#### Actors Involved:

- End User: Anna Koper (example user)
- System: Foodies application

#### Preconditions:

- The user is logged into their account on the Foodies platform.
- The recipe substitution functionality is enabled and accessible.
- The user has at least one recipe available to modify.

#### Test Steps:

1. Substitute an Ingredient: Navigate to a recipe (e.g., Chivito Uruguayo) and verify recipe details.
  - Click on the arrow near an ingredient to initiate substitution.
  - Select a substitute (e.g., gluten-free bread) and confirm.
  - Save the substitution and check whether the modification is saved in My Modifications.

| kcal: 1059   protein: 86 g   fat: 65 g   carbs: 19 g                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <h4>Ingredients</h4> <ul style="list-style-type: none"> <li>100 grams of ham</li> <li>100 grams of mozzarella</li> <li>100 grams of bacon</li> <li>20 grams of beef brisket</li> <li>20 grams of gluten-free bread</li> <li>10 grams of lettuce</li> <li>10 grams of tomato</li> <li>10 grams of egg</li> <li>10 grams of onion</li> <li>10 grams of pepper</li> </ul> | <h4>Steps</h4> <p>Crush the meat so that it is finite and we put it on a griddle to brown. Put the eggs, bacon and ham to fry. Cut the bread in half, put the beef brisket, the fried eggs, the bacon, the ham, the mozzarella, the tomato and the lettuce. Cover with the other half of the bread and serve.</p> |

Fig. 7.9. Saved Substitutions

#### 2. Edge Cases: Attempt to substitute an ingredient that has no available alternatives.

- Expected Result: There is no arrow near these ingredients.



Fig. 7.10. Ingredients without substitutes

Expected Results:

- Users can substitute ingredients easily from a list of compatible options.
- The system reflects updated ingredient details and nutritional values after substitution.
- Users can revert substitutions without issues.
- Substitution suggestions are contextually appropriate.

## 8. SUMMARY

Recognizing the limitations of existing platforms and the significant impact of dietary restrictions, we believe our web application offers a compelling solution for individuals with severe allergies or strict dietary needs. By facilitating effortless product substitutions, our platform empowers users to maintain a healthy and balanced diet while enjoying their favorite foods. This is particularly impactful for those with allergies, as it helps them avoid potentially life-threatening health risks. Ultimately, our goal is to make dietary restrictions less restrictive for all users, enabling them to savor their preferred meals without compromise. This driving motivation has fueled our development efforts.

Working on this project has provided our team with invaluable experience and knowledge beyond the technical aspects of implementation and technology utilization. We have gained crucial insights into project planning, design, and the vital importance of effective team communication and task division in today's professional environment. Our project journey encompassed three distinct phases: business analysis, implementation and verification, and finally, testing and validation. During the initial business analysis phase, we conducted thorough market research to assess the need for such an application and identify any existing competitors. Upon confirming a market gap, we proceeded to gather user requirements through the creation of user personas and a functional prototype. We also established a product backlog and adopted the Agile Scrum methodology to guide the subsequent development stages. The implementation phase involved selecting a suitable development methodology, building the necessary database, and embarking on a series of two-week sprints. Following the successful completion of the implementation phase, rigorous testing was conducted to ensure the application's functionality and user experience. These comprehensive testing efforts concluded the project with satisfactory results.

Our project could also offer other functionalities to consider for application development. For example, we could enhance user functionality by adding features such as the ability to flag disliked ingredients. Another valuable addition would be allergy-based filtering, allowing users to automatically swap allergens in recipes. Additionally, the implementation of a recommendation system could significantly improve the user experience by suggesting recipes based on individual preferences. Advanced filtering methods could also be introduced that could dynamically adjust ingredients in recipes to match the desired diet. Finally, the inclusion of an admin interface and dynamic streams would provide better system management and real-time updates. These improvements would not only enhance the user experience, but also improve the administration of the application.



## REFERENCES

- [1] Christopher M. Warren, Jialing Jiang, and Ruchi S. Gupta. "Epidemiology and Burden of Food Allergy". In: *Current Allergy and Asthma Reports* 20.2 (Feb. 2020), p. 6. ISSN: 1534-6315. DOI: 10.1007/s11882-020-0898-7. URL: <https://doi.org/10.1007/s11882-020-0898-7>.
- [2] Charlotte B. Martin et al. *Prescription Drug Use in the United States, 2015–2016*. NCHS Data Brief 389. Hyattsville, MD: National Center for Health Statistics, 2020. URL: <https://www.cdc.gov/nchs/products/databriefs/db389.htm>.
- [3] Foodiversity. *Foodiversity: A Resource for Food Diversity and Nutrition Education*. Accessed: 2024-11-11. 2024. URL: <https://foodiversity.org/>.
- [4] L. Rappoport. *How We Eat: Appetite, Culture, and the Psychology of Food*. Ecw Press, 2010. ISBN: 9781554902415. URL: [https://books.google.pl/books?id=srj\\_y0b8Mh8C](https://books.google.pl/books?id=srj_y0b8Mh8C).
- [5] Heather Haddon. *Gen Z Loves the Gym. That's a Big Problem for Gyms*. Accessed: 2024-11-12. 2024. URL: <https://www.businessinsider.nl/gen-z-loves-the-gym-thats-a-big-problem-for-gyms/>.
- [6] Fortune Well Staff. *Gym usage is soaring post-pandemic—and it's all thanks to Gen Z*. Accessed: 2024-11-12. 2024. URL: <https://fortune.com/well/article/gym-use-increase-pandemic-gen-z/>.
- [7] World Health Organization. *In brief: Priority food allergens*. Tech. rep. Accessed: 2024-11-13. World Health Organization, 2023. URL: <https://iris.who.int/bitstream/handle/10665/378476/B09009-eng.pdf?sequence=1>.
- [8] Food Allergy Research Education. *Food Allergy Facts and Statistics*. Accessed: 2024-11-12. 2024. URL: <https://www.foodallergy.org/resources/facts-and-statistics>.
- [9] Generation Vegan. *Why Do People Go Vegan?* Accessed: November 14, 2024. 2024. URL: <https://genv.org/why-do-people-go-vegan/>.
- [10] Kristof Dhont and Maria Ioannidou. *Vegan for the Animals*. Author's Accepted Manuscript. 2021. URL: <https://kar.kent.ac.uk/id/eprint/91380>.
- [11] Mayo Clinic Staff. *Weight Loss: Choosing a Diet That's Right for You*. Accessed: 2024-11-12. 2024. URL: <https://www.mayoclinic.org/healthy-lifestyle/weight-loss/in-depth/weight-loss/art-20044318>.
- [12] Healthdirect Australia. *Ketogenic Diet*. Accessed: 2024-11-12. 2024. URL: <https://www.healthdirect.gov.au/ketogenic-diet>.
- [13] Tasteaholics. *Keto-Friendly Food Swaps for Common Ingredients*. Accessed: 2024-11-24. 2024. URL: <https://www.tasteaholics.com/guides-and-faqs/keto-friendly-food-swaps-for-common-ingredients/>.
- [14] KetoDiet App. *The Ultimate Keto Food Swap List*. Accessed: 2024-11-24. 2024. URL: <https://ketodietapp.com/Blog/lchf/the-ultimate-keto-food-swap-list>.

- [15] U.S. Department of Agriculture, Food Safety and Inspection Service. *Food Allergies: The Big 9*. Accessed: 2024-11-12. 2024. URL: <https://www.fsis.usda.gov/food-safety/safe-food-handling-and-preparation/food-safety-basics/food-allergies-big-9>.
- [16] Celiac Disease Foundation. *What is Gluten?* Accessed: November 14, 2024. 2024. URL: <https://celiac.org/gluten-free-living/what-is-gluten/>.
- [17] Beyond Celiac. *Gluten-Free Diet*. Accessed: November 14, 2024. 2024. URL: <https://www.beyondceliac.org/gluten-free-diet/>.
- [18] Cook Smarts. *Gluten-Free Diet Substitutions List*. Accessed: November 14, 2024. 2024. URL: <https://www.cooksmarts.com/articles/gluten-free-diet-substitutions-list/>.
- [19] Activate Foods. *Gluten-Free Staples*. Accessed: November 14, 2024. 2024. URL: <https://www.activatefoods.com.au/blogs/news/gluten-free-staples>.
- [20] Arizona Digestive Health. *Lactose-Free Diet*. Accessed: November 14, 2024. 2024. URL: <https://arizonadigestivehealth.com/gastroenterology-diets/lactose-free-diet>.
- [21] BBC Good Food. *Spotlight on Dairy-Free*. Accessed: November 14, 2024. 2024. URL: <https://www.bbcgoodfood.com/health/special-diets/spotlight-dairy-free>.
- [22] Healthline. *Lactose-Free Diet*. Accessed: November 14, 2024. 2024. URL: <https://www.healthline.com/nutrition/lactose-free-diet#target-group>.
- [23] Athanasios Lamprou, Dimitra Vagiona. *International Journal of Real Estate and Land Planning*. 2018. URL: <https://core.ac.uk/download/pdf/267933948.pdf>.
- [24] Ali Yahya Ghani, Yusmadi Yah Jusoh, Marzanah A. Jabar and Norhayati Mohd Ali. *The Critical Success Factors (CSFs) for IT Projects*. 2024. URL: <https://jtec.utem.edu.my/jtec/article/view/2865/2004>.
- [25] Thomas A. Alspaugh, Susan Elliott Sim, Kristina Winbladh, Mamadou H. Diallo , Leila Naslavsky, Hadar Ziv, Debra J. Richardson. *The Importance of Clarity in Usable Requirements Specification Formats*. 2006. URL: <https://citeseerx.ist.psu.edu/document?%20repid=rep1&type=pdf&doi=88a02d6061a0f2d717617e770f22ec0f23128940>.
- [26] Hubert F. Hofmann, Franz Lehner. *Requirements Engineering as a Success Factor in Software Projects*. 2001. URL: <https://ics.uci.edu/~wscacchi/SA/Readings/Req-Engr-%20SuccessFactors-Software-July01.pdf>.
- [27] Statista Research Department. *Leading reasons for software project failure according to developers worldwide, as of 2015*. 2015. URL: <https://www.statista.com/statistics/627648/worldwide-%20software-developer-survey-project-failure/>.
- [28] Ashok Sasidharan Nair. *Requirements Analysis: Methods and Best Practices*. 2023. URL: <https://ashok-s-nair.medium.com/engineering-requirements-%20analysis-methods-and-best-practices-6e0c7cc9ddfa>.
- [29] Tousif ur Rehman, Muhammad Naeem Ahmed Khan, Naveed Riaz. *Analysis of Requirement Engineering Processes, Tools/Techniques and Methodologies*. 2013. URL: <https://www.mecspress.org/ijitcs/ijitcs-v5-n3/IJITCS-V5-N3-5.pdf>.

- [30] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn E.M. van der Werf, Sjaak Brinkkemper. *The Use and Effectiveness of User Stories in Practice*. 2020. URL: <https://webpace.science.uu.nl/~dalpi001/papers/luca-dalp-werf-brin-16-refsq.pdf>.
- [31] Franz Lang, Alexander Mjöberg. *Prototyping as a Requirements Engineering Technique*. 2020. URL: <https://lup.lub.lu.se/luur/download?func=downloadFile&recordId=9024946&fileId=9024947>.
- [32] Anna Grigoryan. *What Are The Types of Product Backlog Items?* URL: <https://www.wrike.com/product-management-guide/faq/what-are-the-types-of-product-backlog-items/>.
- [33] TheMealDB. 2024. URL: <https://www.themealdb.com/>.
- [34] mealdb.com. *FAQ - mealdb*. 2024. URL: <https://www.themealdb.com/faq.php>.
- [35] Syndigo LLC. 2024. URL: <https://www.nutritionix.com/>.
- [36] Holistics Software. *dbdiagram.io*. 2024. URL: <https://dbdiagram.io/home>.
- [37] Kenneth Reitz. *Requests python library*. 2024. URL: <https://pypi.org/project/requests/>.
- [38] Alexander Albrecht Felix Naumann. *Managing ETL Processes*. 2009. URL: [https://www.researchgate.net/publication/228347220\\_Managing\\_ETL\\_processe](https://www.researchgate.net/publication/228347220_Managing_ETL_processe).
- [39] Python Software Foundation. *Built-in Types*. 2024. URL: <https://docs.python.org/3/library/stdtypes.html>.
- [40] *Cloud SQL for PostgreSQL documentation*. 2024. URL: <https://cloud.google.com/sql/docs/postgres>.
- [41] The Psycopg Team Federico Di Gregorio Daniele Varrazzo. *The cursor class*. 2021. URL: <https://www.psycopg.org/docs/cursor.html>.
- [42] SmartBear Software. *OpenAPI Specification Version 3.1.0*. 2024. URL: <https://swagger.io/specification/>.
- [43] *Angular Official Documentation*. URL: <https://angular.dev/overview>.
- [44] *RxJS official documentation*. URL: <https://rxjs.dev/guide/overview>.
- [45] Broadcom. *Cloud SQL for PostgreSQL documentation*. 2024. URL: <https://docs.spring.io/spring-data/jpa/reference/index.html>.
- [46] *Redis Official Documentation*. URL: <https://redis.io/docs/latest/>.
- [47] IBM. *What is containerization?* URL: <https://www.ibm.com/topics/containerization>.
- [48] Aidan Sayers Ian Miell. *Docker in Practice, Second Edition*. MANNING PUBLICATIONS, 2019. ISBN: 9781617294808.
- [49] Igor Sysoev. *Nginx*. URL: <https://nginx.org/en/>.
- [50] *App Engine*. 2024. URL: <https://cloud.google.com/appengine?hl=en>.
- [51] *Memorystore for Redis Cluster overview*. 2022. URL: <https://cloud.google.com/memorystore/docs/cluster/memorystore-for-redis-cluster-overview>.
- [52] *Artifact Registry overview*. 2022. URL: <https://cloud.google.com/artifact-registry/docs/overview>.

- [53] Stefan Bechtold, Sam Brannen, Johannes Link, Matthias Merdes, Marc Philipp, Juliette de Rancourt, Christian Stein. *JUnit 5 User Guide*. 2024. URL: <https://junit.org/junit5/docs/current/user-guide/>.
- [54] Szczepan Faber, Brice Dutheil, Rafael Winterhalter, Tim van der Lippe, Marcin Grzejszczak, Marcin Zajęczkowski. *Tasty mocking framework for unit tests in Java*. 2024. URL: <https://site.mockito.org/>.
- [55] Marc R. Hoffmann, Brock Janiczak, Evgeny Mandrikov. *JaCoCo Java Code Coverage Library*. 2024. URL: <https://www.eclemma.org/jacoco/>.
- [56] Gem Kaner. *An Introduction to Scenario Testing*. URL: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=31a3d8676911e943c346f777d8d68a2b60711024>.
- [57] Ruchi S. Gupta Christopher M. Warren Jialing Jiang. "Epidemiology and Burden of Food Allergy". In: *CoRR* abs/1810.01109 (2018). arXiv: 1810.01109. URL: <http://arxiv.org/abs/1810.01109>.
- [58] Food Is Good. *Navigating Dairy-Free Choices: 45 Ingredients to Avoid*. Accessed: November 14, 2024. 2024. URL: <https://foodisgood.com/blog/post/navigating-dairy-free-choices-45-ingredients-to-avoid/>.
- [59] *Cloud SQL for PostgreSQL documentation*. 2024. URL: <https://cloud.google.com/iam/docs/service-account-overview>.

## **LIST OF DRAWINGS**

## LIST OF TABLES

# Product Backlog

| Name                               | Status | Assignee         | Description                                                                                                                                 | Priority     | Sprint      | Story point estimate |
|------------------------------------|--------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------|-------------|----------------------|
| Create prototype                   | Done   | Agnieszka Kuleta | Create prototype with basic functionalities using figma.                                                                                    | Should have  | CP Sprint 1 | 5                    |
| Migrate DB to GCP                  | Done   | Kamil Dędzia     | Migrate whole DB to PostgreSQL service on GCP.                                                                                              | Nice to have | CP Sprint 5 | 3                    |
| Prototype                          | Done   | Agnieszka Kuleta | Create prototype of the application with basic functionalities using figma.                                                                 | Should have  | CP Sprint 2 | 4                    |
| Add flags recipes and ingredients  | Done   | Kamil Dędzia     | Flag all the recipes to which diet type and what type of meal is it. Also flag the ingredients whether they are suitable for specific diet. | Should have  | CP Sprint 4 | 5                    |
| Persist ingredients micro elements | Done   | Kamil Dędzia     | Add fats, proteins, carbohydrates and total kcal for each ingredients using chosen API.                                                     | Should have  | CP Sprint 3 | 4                    |
| Sorting recipes.                   | Done   | Wiktor Czetyrbok | As a user, I want to sort the results by calories or rating.                                                                                | Nice to have | CP Sprint 5 | 5                    |
| Remembering last sign in.          | Done   | Wiktor Czetyrbok | As a user, I want the service to remember my last sign in on my device.                                                                     | Should have  | CP Sprint 3 | 1                    |
| Adding recipe                      | Done   | Wiktor Czetyrbok | As a user, I want to be able to add new recipe by contacting admin.                                                                         | Should have  | CP Sprint 4 | 3                    |
| Account creation and logging in    | Done   | Wiktor Czetyrbok | As a user, I want to be able to create my personal account and log in.                                                                      | Must have    | CP Sprint 2 | 4                    |
| Create data dictionary             | Done   | Agnieszka Kuleta | Create data dictionary for all the tables in database.                                                                                      | Nice to have | CP Sprint 4 | 2                    |
| Prioritizing requirements          | Done   | Kamil Dędzia     | Add field priority for each issue.                                                                                                          | Should have  | CP Sprint 2 | 2                    |
| Write motivation                   | Done   | Agnieszka Kuleta | Write motivation subchapter.                                                                                                                | Must have    | CP Sprint 2 | 2                    |
| Create basic frontend service      | Done   | Wiktor Czetyrbok | Create basic angular application for culinary views                                                                                         | Must have    | CP Sprint 2 | 3                    |
| Create data documentation          | Done   | Agnieszka Kuleta | Create basic documentation about data used and APIs.                                                                                        | Nice to have | CP Sprint 4 | 3                    |
| Transform and clean the data       | Done   | Kamil Dędzia     | Transform the data in such way it fits the schema, detect any missing values or errors with proper handling.                                | Must have    | CP Sprint 4 | 4                    |

|                                             |      |                  |                                                                                                                                                                                         |              |             |   |
|---------------------------------------------|------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|-------------|---|
| Populate database with the use of API       | Done | Kamil Dęda       | Populate the database with the use of the API and transform required fields.                                                                                                            | Must have    | CP Sprint 2 | 5 |
| Create database schema                      | Done | Kamil Dęda       | Create database schema with the use of the previously created design, using selected database of your own.                                                                              | Must have    | CP Sprint 1 | 4 |
| Create list of substitutes for ingredients. | Done | Agnieszka Kuleta | For selected ingredients create list of substitutes for them. Remember about the proportion of each product.                                                                            | Must have    | CP Sprint 4 | 5 |
| Review comments and rates                   | Done | Wiktor Czetyrbok | As a user, I want to be able to see the average rating score for meals in pleasant manner, and be able to read the comments.                                                            | Should have  | CP Sprint 4 | 3 |
| Change user password                        | Done | Wiktor Czetyrbok | As a user, I want to be able to change my password using my email.                                                                                                                      | Must have    | CP Sprint 2 | 2 |
| Update details in profile section           | Done | Wiktor Czetyrbok | As a user, I want to be able to update my profile with my name, surname, birthdate, email and be able to change it in case of typos or actual changes.                                  | Must have    | CP Sprint 4 | 3 |
| Recipe portal security config               | Done | Wiktor Czetyrbok | Restrict only authorised access to data layer with proper credentials and encryption system.                                                                                            | Should have  | CP Sprint 3 | 3 |
| User portal security config                 | Done | Wiktor Czetyrbok | Configure security system for the culinary portal to ensure the protection of user data and prevent unauthorized access.                                                                | Must have    | CP Sprint 3 | 3 |
| Persist user diet                           | Done | Agnieszka Kuleta | As a user of our culinary platform, I want my diets to be saved so that I can easily access recipes. Along with previously saved recipes                                                | Nice to have | CP Sprint 3 | 3 |
| Set up user profile picture                 | Done | Agnieszka Kuleta | As a registered user of our culinary portal, I want to be able to set up a profile picture to personalize my account.                                                                   | Should have  | CP Sprint 4 | 2 |
| Persist user preferences                    | Done | Wiktor Czetyrbok | As a registered user on our culinary portal, I want my preferences to be saved across sessions, so I can have a personalized experience tailored to my tastes and dietary requirements. | Nice to have | CP Sprint 3 | 3 |
| Allow user to log in                        | Done | Agnieszka Kuleta | As a user I want to log in to culinary portal. Want to access multiple tabs without logging in. I want to have interactive UI which allows easy logging into application.               | Must have    | CP Sprint 2 | 4 |
| Create a project plan                       | Done | Agnieszka Kuleta | Decide over structure of work division.                                                                                                                                                 | Must have    | CP Sprint 1 | 2 |
| Setup CI/CD user portal repository          | Done | Wiktor Czetyrbok | Create github repository, create setup CI /CD tools for services                                                                                                                        | Should have  | CP Sprint 1 | 3 |



|                                      |      |                  |                                                                                                                                                                                        |              |             |   |
|--------------------------------------|------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|-------------|---|
| Create communication channel         | Done | Kamil Dędzia     | Decide over which communicator will be used, add team members with proper access                                                                                                       | Must have    | CP Sprint 1 | 2 |
| Create initial document              | Done | Agnieszka Kuleta | Skeleton of thesis on overleaf in LaTeX form.                                                                                                                                          | Must have    | CP Sprint 1 | 2 |
| Research possible diets              | Done | Agnieszka Kuleta | Search and note possible diets that first version of an app can be based of.                                                                                                           | Should have  | CP Sprint 2 | 3 |
| Recipe Saving                        | Done | Wiktor Czetyrbok | As a user, I want to easily save my favorite recipes for easy access later. And can easily access them later and refer back to them for cooking inspiration or future meal planning.   | Must have    | CP Sprint 3 | 3 |
| Input recipe ratings & reviews       | Done | Agnieszka Kuleta | As a user, I want to rate and review recipes I have tried to help others make informed choices, grade them in scale 1-5.                                                               | Should have  | CP Sprint 3 | 4 |
| Recipe Details                       | Done | Agnieszka Kuleta | As a user, I want to view detailed information about a specific recipe, including ingredients, instructions, and nutritional facts.                                                    | Must have    | CP Sprint 3 | 4 |
| Recipe Filtering                     | Done | Agnieszka Kuleta | As a user, I want to filter recipes based on dietary restrictions or preferences (e.g., vegan, gluten-free), exclude products that I do not like, include products that I want to use. | Should have  | CP Sprint 5 | 4 |
| Recipe Search                        | Done | Wiktor Czetyrbok | As a user, I want to search for specific recipes based on ingredients, cuisine type or rating. I would like to see other users recipes and recommendations.                            | Nice to have | CP Sprint 5 | 5 |
| Recipe Browsing                      | Done | Wiktor Czetyrbok | As a user, I want to browse through various culinary recipes available on the portal and get all the information about it.                                                             | Should have  | CP Sprint 3 | 3 |
| User Registration                    | Done | Wiktor Czetyrbok | As a new user, I want to register an account on the culinary portal. I want to do it via simple and interactive UI. I want my private data to be stored safely and encrypted           | Must have    | CP Sprint 2 | 3 |
| Create github organization           | Done | Wiktor Czetyrbok | Create github organization, add members with correct accesses                                                                                                                          | Must have    | CP Sprint 1 | 2 |
| Persist ingredients to database      | Done | Kamil Dędzia     | Persist the basic information about the recipes and ingredients in the database.                                                                                                       | Must have    | CP Sprint 2 | 5 |
| Create basic user portal application | Done | Wiktor Czetyrbok | Create blueprint of an application ,no specific implementation, app skeleton                                                                                                           | Must have    | CP Sprint 2 | 3 |
| Design database schema               | Done | Kamil Dędzia     | Design relational database schema with the use of existing tools, including all the data types.                                                                                        | Must have    | CP Sprint 2 | 3 |
| Decide about technology stack        | Done | Wiktor Czetyrbok | Discuss which technologies are to be used.                                                                                                                                             | Must have    | CP Sprint 1 | 3 |

|                              |      |                  |                                                                        |              |             |   |
|------------------------------|------|------------------|------------------------------------------------------------------------|--------------|-------------|---|
| Create user persona          | Done | Agnieszka Kuleta | Create personas to better understand the needs.                        | Nice to have | CP Sprint 1 | 4 |
| Research culinary recipe API | Done | Kamil Dędzia     | Check the copyrights and test API responses and structure of the data. | Must have    | CP Sprint 1 | 4 |

# Data Dictionary

## Description of entities:

| Entity 1: recipe                                                                                                   |             |         |                                                                                                    |                                         |
|--------------------------------------------------------------------------------------------------------------------|-------------|---------|----------------------------------------------------------------------------------------------------|-----------------------------------------|
| Contains information about recipe which are gathered in our service. An entry is added when a new recipe is added. |             |         |                                                                                                    |                                         |
| Attributes                                                                                                         |             |         |                                                                                                    |                                         |
| Name                                                                                                               | Primary key | Type    | Domain                                                                                             | Description                             |
| recipe_id                                                                                                          | yes         | integer | Number from 1 to 10000                                                                             | Unique recipe identifier.               |
| general_recipe_id                                                                                                  | no          | Integer | Number from 1 to 10000                                                                             | Reference to general recipe identifier. |
| name                                                                                                               | no          | text    | Up to 50 chars.<br>Roman alphabet with polish signs, any special characters not allowed(ex. @,!,_) | Name of recipe.                         |
| description                                                                                                        | no          | text    | Up to 500 chars.<br>Roman alphabet with polish signs, special characters allowed.                  | Description of the recipe.              |
| diet_type_id                                                                                                       | no          | integer | Number from 1 to 100                                                                               | Reference to diet_type identifier.      |

| Entity 2: general_recipe                                                                                                              |             |         |                                                                                                    |                                                |
|---------------------------------------------------------------------------------------------------------------------------------------|-------------|---------|----------------------------------------------------------------------------------------------------|------------------------------------------------|
| Contains information about recipe. An entry is added when a new recipe is added which general version does not exist in the database. |             |         |                                                                                                    |                                                |
| Attributes                                                                                                                            |             |         |                                                                                                    |                                                |
| Name                                                                                                                                  | Primary key | Type    | Domain                                                                                             | Description                                    |
| general_recipe_id                                                                                                                     | yes         | integer | Number from 1 to 10000                                                                             | Unique recipe identifier.                      |
| name                                                                                                                                  | no          | text    | Up to 50 chars.<br>Roman alphabet with polish signs, any special characters not allowed(ex. @,!,_) | Name of recipe.                                |
| photo_url                                                                                                                             | no          | text    | Roman alphabet without polish signs, special characters not allowed.                               | Link to photo of the dish.                     |
| meal_type                                                                                                                             | no          | text    | One from list:<br>'BREAKFAST',<br>'LUNCH', 'DINNER',<br>'SUPPER', 'DESSERT'                        | Type of the meal depending on the time of day. |
| description                                                                                                                           | no          | text    | Up to 500 chars.                                                                                   | Description of the recipe.                     |

|                |    |         |                                                                                   |                                        |
|----------------|----|---------|-----------------------------------------------------------------------------------|----------------------------------------|
|                |    |         | Roman alphabet with polish signs, special characters allowed.                     |                                        |
| steps          | no | text    | Up to 500 chars.<br>Roman alphabet with polish signs, special characters allowed. | Instructions on how to prepare a meal. |
| base_recipe_id | no | integer | Number from 1 to 10000.                                                           | Reference to recipe identifier.        |

| Entity 3: diet_type              |             |         |                                                                                                    |                        |
|----------------------------------|-------------|---------|----------------------------------------------------------------------------------------------------|------------------------|
| Contains information about diet. |             |         |                                                                                                    |                        |
| Attributes                       |             |         |                                                                                                    |                        |
| Name                             | Primary key | Type    | Domain                                                                                             | Description            |
| diet_type_id                     | yes         | integer | Number from 1 to 100.                                                                              | Unique diet identifier |
| diet_type                        | no          | text    | Up to 20 chars.<br>Roman alphabet with polish signs, any special characters not allowed(ex. @,!,_) | Name of diet.          |

| Entity 4: contains                                                                                                                                            |             |         |                                      |                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|---------|--------------------------------------|-------------------------------------------------------|
| Contains information about recipe's all ingredients, which are individual case for each recipe. An entry is added when the new ingredient is added to recipe. |             |         |                                      |                                                       |
| Attributes                                                                                                                                                    |             |         |                                      |                                                       |
| Name                                                                                                                                                          | Primary key | Type    | Domain                               | Description                                           |
| contains_id                                                                                                                                                   | yes         | integer | Number from 1 to 10000.              | Unique contains identifier.                           |
| Amount                                                                                                                                                        | no          | float   | Number from 0 up to 1000.            | Amount of unit needed                                 |
| Measure                                                                                                                                                       | no          | text    | Number from 0 to 1000, n/a if empty. | Indicates the measurement of the given unit in grams. |
| recipe_id                                                                                                                                                     | no          | number  | Number from 1 to 10000.              | Reference to recipe identifier.                       |
| ingredient_id                                                                                                                                                 | no          | number  | Number from 1 to 10000.              | Reference to ingredient identifier.                   |

| Entity 5: ingredient                                                                                                                         |  |  |  |  |
|----------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|--|
| Number of ingredients is up to 10000.Contains information about ingredient used in recipe. An entry is added when a new ingredient is added. |  |  |  |  |
| Attributes                                                                                                                                   |  |  |  |  |

| Name            | Primary key | Type    | Domain                                                                                             | Description                                  |
|-----------------|-------------|---------|----------------------------------------------------------------------------------------------------|----------------------------------------------|
| id_ingredient   | yes         | Integer | Number up to 10000.                                                                                | Unique ingredient identifier.                |
| name            | no          | text    | Up to 50 chars.<br>Roman alphabet with polish signs, any special characters not allowed(ex. @,!,_) | Name of ingredient                           |
| fat             | no          | Float   | Float from 0 up to 100.                                                                            | Amount of fat for 100g.                      |
| protein         | no          | Float   | Float from 0 up to 100.                                                                            | Amount of protein for 100g.                  |
| carbohydrate    | no          | Float   | Float from 0 up to 100.                                                                            | Amount of carbohydrates for 100g.            |
| kcal            | no          | Float   | Float from 0 up to 900.                                                                            | Amount of kcal for 100g.                     |
| is_vegan        | no          | Binary  | Contain 0(if not) and 1 if (yes)                                                                   | Indicates whether it is vegan or not.        |
| is_vegetarian   | no          | Binary  | Contain 0(if not) and 1 if (yes)                                                                   | Indicates whether it is vegetarian or not.   |
| is_gluten_free  | no          | Binary  | Contain 0(if not) and 1 if (yes)                                                                   | Indicates whether it is gluten free or not.  |
| is_lactose_free | no          | Binary  | Contain 0(if not) and 1 if (yes)                                                                   | Indicates whether it is lactose free or not. |
| is_keto         | no          | Binary  | Contain 0(if not) and 1 if (yes)                                                                   | Indicates whether it is keto or not.         |

| Entity 6: substitute                                        |             |         |                     |                                     |
|-------------------------------------------------------------|-------------|---------|---------------------|-------------------------------------|
| Contains information about substitutes of given ingredient. |             |         |                     |                                     |
| Attributes                                                  |             |         |                     |                                     |
| Name                                                        | Primary key | Type    | Domain              | Description                         |
| substitute_id                                               | yes         | integer | Number up to 10000. | Unique substitute identifier.       |
| ingredient1_id                                              | no          | integer | Number up to 10000. | Reference to ingredient identifier. |
| ingredient2_id                                              | no          | integer | Number up to 10000. | Reference to ingredient identifier. |

| Entity 7: specific                                       |             |      |        |             |
|----------------------------------------------------------|-------------|------|--------|-------------|
| Contains information about specific preferences of user. |             |      |        |             |
| Attributes                                               |             |      |        |             |
| Name                                                     | Primary key | Type | Domain | Description |

|               |     |         |                                  |                                            |
|---------------|-----|---------|----------------------------------|--------------------------------------------|
| specific_id   | yes | integer | Number up to 10000.              | Unique specific preference identifier      |
| user_id       | no  | integer | Number up to 1000.               | Reference to user identifier.              |
| ingredient_id | no  | integer | Number up to 10000.              | Reference to ingredient identifier.        |
| likes         | No  | Binary  | Contain 0(if not) and 1 if (yes) | Indicates whether the user likes it or not |

| Entity 8: review                                                                  |             |         |                                                                                                     |                                                    |
|-----------------------------------------------------------------------------------|-------------|---------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Contains information about reviews, which is published by user on a given recipe. |             |         |                                                                                                     |                                                    |
| Attributes                                                                        |             |         |                                                                                                     |                                                    |
| Name                                                                              | Primary key | Type    | Domain                                                                                              | Description                                        |
| review_id                                                                         | yes         | integer | Number up to 10000.                                                                                 | Unique specific preference identifier              |
| user_id                                                                           | no          | integer | Number up to 10000.                                                                                 | Reference to user identifier.                      |
| general_recipe_id                                                                 | no          | integer | Number up to 10000.                                                                                 | Reference to general_recipe identifier.            |
| rating                                                                            | No          | Integer | Number from 1 to 5                                                                                  | Indicates the degree of liking the recipe by user. |
| opinion                                                                           | No          | Text    | Up to 500 signs.<br>Roman alphabet with polish signs, any special characters not allowed(ex. @,!,_) | Describes the experience of recipe.                |

| Entity 9: user                    |             |             |                                                                                                    |                        |
|-----------------------------------|-------------|-------------|----------------------------------------------------------------------------------------------------|------------------------|
| Contains information about users. |             |             |                                                                                                    |                        |
| Attributes                        |             |             |                                                                                                    |                        |
| Name                              | Primary key | Type/domain | Domain                                                                                             | Description            |
| user_id                           | Yes         | integer     | Number from 1 to 10000                                                                             | Unique user identifier |
| name                              | no          | text        | Up to 20 chars.<br>Roman alphabet with polish signs, any special characters not allowed(ex. @,!,_) | Name of user           |
| email                             | No          | Text        | Roman alphabet up to 50 chars, must contain @ in between 2 strings.                                | Email of user          |
| password                          | No          | Text        | Minimum 5 and up to 20 chars.<br>Roman alphabet with polish signs, any special                     | Password of user       |

|                     |    |        |                                                                                                                          |                                             |
|---------------------|----|--------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
|                     |    |        | characters are allowed(ex. @,!,_), but not required.                                                                     |                                             |
| birthdate           | no | date   | Allows to introduce date from the interval (1.01.1900-today)<br>The form: ('YYYYMMDD')                                   | Birthdate of the user.                      |
| create_date         | no | date   | The user's account creation date. Allows to introduce date from the interval (1.01.2024-today)<br>The form: ('YYYYMMDD') | Date of the creation of account.            |
| pref_is_vegan       | No | Binary | Contain 0(if not) and 1 if (yes)                                                                                         | Specification of being vegan or not.        |
| pref_is_gluten_free | No | Binary | Contain 0(if not) and 1 if (yes)                                                                                         | Specification of eating gluten-free or not. |
| account_enabled     | No | Binary | Contain 0(if not) and 1 if (yes)                                                                                         | Information whether enabled.                |
| account_expired     | No | Binary | Contain 0(if not) and 1 if (yes)                                                                                         | Information whether expired.                |
| account_locked      | No | Binary | Contain 0(if not) and 1 if (yes)                                                                                         | Information whether locked.                 |
| credentials_expired | No | Binary | Contain 0(if not) and 1 if (yes)                                                                                         | Information whether credentials expired.    |

| Entity 10: favourites                                 |             |         |                         |                                         |
|-------------------------------------------------------|-------------|---------|-------------------------|-----------------------------------------|
| Contains information about user's favourites recipes. |             |         |                         |                                         |
| Attributes                                            |             |         |                         |                                         |
| Name                                                  | Primary key | Type    | Domain                  | Description                             |
| favourite_id                                          | yes         | integer | Number 1 from to 10000. | Unique favourites identifier            |
| user_id                                               | no          | integer | Number 1 from to 10000. | Reference to user identifier.           |
| general_recipe_id                                     | no          | integer | Number 1 from to 10000. | Reference to general_recipe identifier. |

| Entity 11: modified_recipe                         |             |         |                         |                                       |
|----------------------------------------------------|-------------|---------|-------------------------|---------------------------------------|
| Contains the relationship between recipe and user. |             |         |                         |                                       |
| Attributes                                         |             |         |                         |                                       |
| Name                                               | Primary key | Type    | Domain                  | Description                           |
| modified_recipe_id                                 | yes         | integer | Number 1 from to 10000. | Unique identifier od modified_recipe. |
| user_id                                            | no          | integer | Number 1 from to 10000. | Reference to user identifier.         |
| recipe_id                                          | no          | integer | Number 1 from to 10000. | Reference to recipe identifier.       |

| Entity 12: user_diet_type                                |             |         |                         |                                    |
|----------------------------------------------------------|-------------|---------|-------------------------|------------------------------------|
| Contains the relationship between diet_type_id and user. |             |         |                         |                                    |
| Attributes                                               |             |         |                         |                                    |
| Name                                                     | Primary key | Type    | Domain                  | Description                        |
| user_id                                                  | yes         | integer | Number 1 from to 10000. | Reference to user identifier.      |
| diet_type_id                                             | yes         | integer | Number 1 from to 10000. | Reference to diet_type identifier. |

| Entity 13: roles                              |             |      |        |             |
|-----------------------------------------------|-------------|------|--------|-------------|
| Contains the information about the roles and. |             |      |        |             |
| Attributes                                    |             |      |        |             |
| Name                                          | Primary key | Type | Domain | Description |



|         |     |         |                                                                                                 |                               |
|---------|-----|---------|-------------------------------------------------------------------------------------------------|-------------------------------|
| role_id | yes | integer | Number 1 from to 10000.                                                                         | Unique identifier.            |
| user_id | no  | integer | Number 1 from to 10000.                                                                         | Reference to user identifier. |
| role    | no  | text    | Up to 20 chars. Roman alphabet with polish signs, any special characters not allowed(ex. @,!,_) | Name of a role.               |

## Relationships

| ID | Name                     | Table 1        | Table 2        | Type | Description                                         |
|----|--------------------------|----------------|----------------|------|-----------------------------------------------------|
| 1  | user_diet_type_user      | users          | user_diet_type | 1..n | Defines the diet preferences of each user.          |
| 2  | user_diet_type_diet_type | diet_type      | user_diet_type | 1..n | Categorizes users based on specific dietary types.  |
| 3  | user_roles               | users          | roles          | 1..n | Assigns roles to users for system functionalities.  |
| 4  | general_recipe_recipe    | general_recipe | recipe         | 1..n | Links general recipes to their detailed versions.   |
| 5  | diet_type_recipe         | diet_type      | recipe         | 1..n | Associates recipes with compatible dietary types.   |
| 6  | recipe_contains          | recipe         | contains       | 1..n | Specifies the ingredients required for each recipe. |

|    |                        |                |                 |      |                                                                |
|----|------------------------|----------------|-----------------|------|----------------------------------------------------------------|
| 7  | contains_ingredient    | contains       | ingredient      | 1..1 | Details the nutritional composition of ingredients.            |
| 8  | substitute_ingredient1 | ingredient     | substitute      | 1..n | Allows substitution of ingredients in recipes.                 |
| 9  | substitute_ingredient2 | ingredient     | substitute      | 1..n | Defines alternative ingredients for specific recipes.          |
| 10 | review_user            | users          | review          | 1..n | Captures user feedback and ratings for recipes.                |
| 11 | review_general_recipe  | general_recipe | review          | 1..n | Collects reviews for individual general recipes.               |
| 12 | favorite_user          | users          | favorite        | 1..n | Tracks the favorite recipes of users.                          |
| 13 | favorite_recipe        | general_recipe | favorite        | 1..n | Shows which recipes are marked as favorite by users.           |
| 14 | modified_user          | users          | modified_recipe | 1..n | Tracks customized recipes created by users.                    |
| 15 | modified_recipe        | recipe         | modified_recipe | 1..n | Maps custom recipes to original recipes.                       |
| 16 | specific_user          | users          | specific        | 1..n | Stores user preferences for specific ingredients.              |
| 17 | specific_ingredient    | ingredient     | specific        | 1..n | Indicates user preferences for or against certain ingredients. |
| 18 | recipe_general_recipe  | recipe         | general_recipe  | 1..1 | Links a recipe to its base , general version.                  |

## RDB schema

**diet\_type(diet\_type\_id, diet\_type)**

- **Primary Key:** (diet\_type\_id)

**general\_recipe(general\_recipe\_id, name, photo\_url, meal\_type, description, steps, base\_recipe\_id)**

- **Primary Key:** (general\_recipe\_id)
- **Foreign Key:** (base\_recipe\_id) REFERENCES recipe(recipe\_id)

**users(user\_id, user\_name, email, password, birthdate, create\_date, photo\_url, pref\_is\_vegan, pref\_is\_gluten\_free, account\_enabled, account\_expired, account\_locked, credentials\_expired)**

- **Primary Key:** (user\_id)

**ingredient(ingredient\_id, name, fat, protein, carbohydrate, kcal, is\_vegan, is\_vegetarian, is\_gluten\_free, is\_lactose\_free, is\_keto)**

- **Primary Key:** (ingredient\_id)

**recipe(recipe\_id, general\_recipe\_id, name, description, diet\_type\_id)**

- **Primary Key:** (recipe\_id)
- **Foreign Key:** (general\_recipe\_id) REFERENCES general\_recipe(general\_recipe\_id)
- **Foreign Key:** (diet\_type\_id) REFERENCES diet\_type(diet\_type\_id)

**contains(contains\_id, amount, measure, recipe\_id, ingredient\_id)**

- **Primary Key:** (contains\_id)
- **Foreign Key:** (recipe\_id) REFERENCES recipe(recipe\_id)
- **Foreign Key:** (ingredient\_id) REFERENCES ingredient(ingredient\_id)

**substitute(substitute\_id, ingredient1\_id, ingredient2\_id)**

- **Primary Key:** (substitute\_id)
- **Foreign Key:** (ingredient1\_id) REFERENCES ingredient(ingredient\_id)
- **Foreign Key:** (ingredient2\_id) REFERENCES ingredient(ingredient\_id)

**specific(specific\_id, user\_id, ingredient\_id, likes)**

- **Primary Key:** (specific\_id)
- **Foreign Key:** (user\_id) REFERENCES users(user\_id)
- **Foreign Key:** (ingredient\_id) REFERENCES ingredient(ingredient\_id)

**review(review\_id, user\_id, general\_recipe\_id, rating, opinion)**

- **Primary Key:** (review\_id)
- **Foreign Key:** (user\_id) REFERENCES users(user\_id)
- **Foreign Key:** (general\_recipe\_id) REFERENCES general\_recipe(general\_recipe\_id)

**favorite(favorite\_id, user\_id, general\_recipe\_id)**

- **Primary Key:** (favorite\_id)
- **Foreign Key:** (user\_id) REFERENCES users(user\_id)
- **Foreign Key:** (general\_recipe\_id) REFERENCES general\_recipe(general\_recipe\_id)

**roles(role\_id, user\_id, role)**

- **Primary Key:** (role\_id)
- **Foreign Key:** (user\_id) REFERENCES users(user\_id)

**user\_diet\_type(user\_id, diet\_type\_id)**

- **Primary Key:** (user\_id, diet\_type\_id)
- **Foreign Key:** (user\_id) REFERENCES users(user\_id)
- **Foreign Key:** (diet\_type\_id) REFERENCES diet\_type(diet\_type\_id)

**modified\_recipe(modified\_recipe\_id, user\_id, recipe\_id)**

- **Primary Key:** (modified\_recipe\_id)
- **Foreign Key:** (user\_id) REFERENCES users(user\_id)
- **Foreign Key:** (recipe\_id) REFERENCES recipe(recipe\_id)